34 Diffusion Models

A diffusion model is a probabilistic generative model defined through a diffusion process. The diffusion process consists of a forward (diffusion) process and a reverse (denoising) process. The two main families of diffusion models are the denoising diffusion probabilistic model (DDPM) and score matching with Langevin dynamics (SMLD).

In the denoising diffusion probabilistic model (DDPM), the forward process starts from the original data and gradually adds noise to it over multiple steps until it becomes Gaussian white noise. The reverse process starts from Gaussian white noise and gradually removes the noise over the same number of steps to restore the original data. Learning is carried out for each step of the reverse process by training a neural network to predict the noise added at each step of the corresponding forward process, thereby performing denoising effectively—that is, data generation.

Score matching with Langevin dynamics (SMLD) learns and uses the score function corresponding to the reverse process, while the forward process exists implicitly. During learning, noise of varying levels is added to the original data, and a neural network is trained to fit the score function of the data distribution at each noise level. During data generation, the learned neural network is used to sample from the noisy data distribution via Langevin dynamics, progressively reducing the noise level, and finally generating data that follows the same distribution as the original data.

The diffusion process is a continuous-time Markov process that can be characterized by stochastic differential equations. The forward and reverse processes of the diffusion process have mutually corresponding stochastic differential equations. DDPM and SMLD can in fact be viewed as methods for solving the stochastic differential equations of the diffusion process.

Compared with other probabilistic generative models, diffusion models have certain advantages in terms of learning accuracy, realism, and diversity of the generated data. They are currently widely applied to tasks such as image generation, speech generation, and molecular structure generation. Another probabilistic generative model, the flow matching model, is a different but closely related approach to diffusion models and will be discussed in Chapter 35.

Sohl-Dickstein et al. published the foundational algorithm for diffusion models in 2015, Song and Ermon proposed SMLD in 2019, and Ho et al. proposed the widely used DDPM in 2020. In 2020, Song et al. elucidated the relationship between stochastic differential equations and diffusion models.

This chapter is organized as follows: Section 34.1 covers DDPM, Section 34.2 covers SMLD, Section 34.3 gives an overview of their relationship as methods for solving stochastic differential equations, and Section 34.4 introduces their application to image generation.

34.1 Basic Principles

In physics, a diffusion process refers to the random motion of particles in a fluid. A typical example is movement from regions of high concentration to regions of low concentration. For instance, a drop of ink placed in a glass of water first seeps from the surface downward, gradually spreads out over time, and finally dissolves uniformly in the water. This whole process is a diffusion process.

The diffusion model in deep learning is a probabilistic generative method for data inspired by the physical diffusion process. Deep learning diffusion models can be applied to tasks such as image generation, speech generation, and molecular structure generation.

The forward process (also called the diffusion process) is a stochastic process. Given a data sample, at each step of the stochastic process, noise is added to the sample; after sufficiently many steps, the sample becomes Gaussian white noise.

Correspondingly, the reverse process (also called the denoising process) is also a stochastic process. Starting from Gaussian white noise, at each step of the stochastic process, the Gaussian white noise is denoised; after sufficiently many steps, a data sample is obtained. The reverse process is essentially the generation process from Gaussian white noise to data.

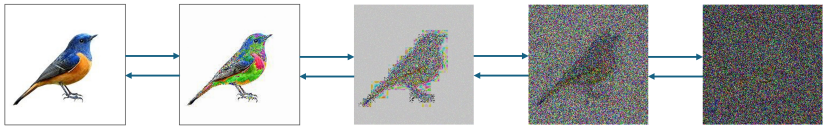

Figure 34.1: Image generation: the forward process gradually transforms an image into Gaussian white noise; the reverse process gradually transforms Gaussian white noise back into an image.

Figure 34.1 illustrates the use of a diffusion model for image generation. Sample data is typically structured (samples are mutually dependent), whereas Gaussian white noise is unstructured (samples are mutually independent and follow a Gaussian distribution). Note: the Gaussian distribution is the probability distribution with maximum entropy given fixed mean and variance (see Chapter 8). The forward process gradually transforms structured data into unstructured data, while the reverse process gradually transforms unstructured data into structured data. The reverse process corresponds to ink that has already diffused in a glass of water flowing backwards against the direction of time to the single drop that was just placed in the water—something that is nearly impossible in the physical world.

Both the forward and reverse processes possess the Markov property, meaning that the state at each time step depends only on the state at the previous time step. This process can be defined in either continuous time or discrete time. Both DDPM and SMLD are built upon discrete-time Markov processes, and their forward and reverse processes each have specific transition probabil-

ity distributions. The forward process is implemented step by step by adding noise, and the reverse process is implemented step by step by removing noise, with each step of the forward process corresponding to a step of the reverse process. The data generation process is the denoising process.

The forward processes of both DDPM and SMLD are predetermined, while the reverse processes are learned from data. The learned models are represented by neural networks that implement one step of denoising. The learning algorithm for DDPM is based on variational inference, and that for SMLD is based on denoising score matching. The denoising or generation processes of both DDPM and SMLD can be understood from the perspective of score matching, which is an essentially important concept for denoising and generation.

DDPM and SMLD were originally proposed as machine learning methods. In a more general setting, the diffusion process can be modeled as a continuous-time Markov process and described by stochastic differential equations. In fact, DDPM and SMLD can be viewed as discrete-time numerical methods for solving different stochastic differential equations. Therefore, stochastic differential equations provide a more general means of modeling the diffusion process.

34.2 Denoising Diffusion Probabilistic Model

This section presents the denoising diffusion probabilistic model (DDPM). We first introduce the definition and properties of the model, and then describe the learning and generation algorithms.

34.2.1 Model Definition and Properties

1. Forward and Reverse Processes

The training data consists of samples drawn from an unknown probability distribution $q(\boldsymbol{x})$, with one sample denoted $\boldsymbol{x}_0$. The goal is to estimate the probability distribution $q(\boldsymbol{x})$ from the training data and generate new data according to the estimated distribution. This is the learning and data generation problem for probabilistic generative models.

Consider the diffusion process, which consists of a forward process and a reverse process. Assume the forward process is a Markov process whose states correspond to samples and whose state transitions correspond to sampling. Starting from the original sample $\boldsymbol{x}_0$, at each step a random sample is drawn according to the transition probability distribution of the Markov process—equivalently, noise is added to the sample—yielding a noisy sample. After T steps, Gaussian white noise $\boldsymbol{x}_T$ is obtained:

$$\boldsymbol{x}_0 \to \boldsymbol{x}_1 \cdots \to \boldsymbol{x}_{t-1} \to \boldsymbol{x}_t \cdots \to \boldsymbol{x}_{T-1} \to \boldsymbol{x}_T$$

where $\boldsymbol{x}_0$ is the original sample and $\boldsymbol{x}_T$ is Gaussian white noise following the standard Gaussian distribution.

Assume the reverse process is also a Markov process whose states correspond to samples and whose state transitions correspond to sampling. Starting from

the Gaussian white noise $\boldsymbol{x}_T$, at each step a random sample is drawn according to the transition probability distribution of the Markov process—equivalently, noise is removed from the sample (denoising)—yielding a denoised sample. After T steps, the original sample $\boldsymbol{x}_0$ is recovered:

$$\boldsymbol{x}_0 \leftarrow \boldsymbol{x}_1 \leftarrow \cdots \boldsymbol{x}_{t-1} \leftarrow \boldsymbol{x}_t \leftarrow \cdots \boldsymbol{x}_{T-1} \leftarrow \boldsymbol{x}_T$$

where $\boldsymbol{x}_T$ is Gaussian white noise and $\boldsymbol{x}_0$ is the original sample. The reverse process is the inverse of the forward process, and each step t ($t = 1, 2, \ldots, T$) of the forward and reverse processes corresponds to each other.

The joint probability distribution of the forward process is

$$q(\boldsymbol{x}_0 \boldsymbol{x}_1 \cdots \boldsymbol{x}_T) = q(\boldsymbol{x}_0) q(\boldsymbol{x}_1|\boldsymbol{x}_0) \cdots q(\boldsymbol{x}_T|\boldsymbol{x}_{T-1}) \tag{34.1}$$

The joint probability distribution of the reverse process is

$$q(\boldsymbol{x}_0 \boldsymbol{x}_1 \cdots \boldsymbol{x}_T) = q(\boldsymbol{x}_T) q(\boldsymbol{x}_{T-1}|\boldsymbol{x}_T) \cdots q(\boldsymbol{x}_0|\boldsymbol{x}_1) \tag{34.2}$$

Consider image data generation. Figure 34.2 shows the sample space of images, where each image is a point in the sample space. The sample space is high-dimensional, but image data typically lies on a low-dimensional manifold, located in the blue region of the figure. In the forward process, the original image is progressively corrupted with noise until it becomes Gaussian white noise, which is primarily located in the green region of the figure. The reverse process is the inverse process: Gaussian white noise is progressively denoised until it becomes an image, which is also the image generation process.

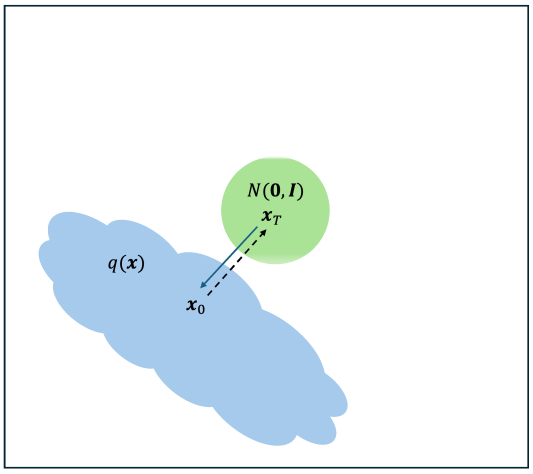

Figure 34.2: The diffusion process in the sample space (see color plate at the front of the book).

2. Forward Process

The forward process of DDPM is the process of gradually transforming the original sample $\boldsymbol{x}_0$ into Gaussian white noise $\boldsymbol{x}_T$. Assuming the transition probability distribution of the forward process is Gaussian—meaning the noise added at each step is Gaussian noise—at step t ($t = 1, 2, \ldots, T$) we have

$$\boldsymbol{x}_t \sim q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = \mathcal{N}(\boldsymbol{x}_t; \sqrt{\alpha_t}\boldsymbol{x}_{t-1}, \beta_t \boldsymbol{I}) \tag{34.3}$$

where the coefficients β_t and α_t satisfy

$$0 < \beta_t < 1$$

$$\alpha_t = 1 - \beta_t$$

An equivalent representation is

$$\boldsymbol{x}_t = \sqrt{\alpha_t}\boldsymbol{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_{t-1} \tag{34.4}$$

where $\boldsymbol{\epsilon}_{t-1}$ follows the standard Gaussian distribution $\boldsymbol{\epsilon}_{t-1} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. Equation (34.4) is called the iteration formula of the DDPM forward process. The first term of the iteration formula is deterministic, and the second term is stochastic.

This makes use of the following property of multivariate Gaussian distributions: if a random variable $\boldsymbol{x}$ follows the isotropic Gaussian distribution $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2\boldsymbol{I})$, then $\boldsymbol{x} = \boldsymbol{\mu} + \sigma\boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$.

Both the density-function-based and the random-variable-based representations have their convenient uses; we will choose between them as needed. In the learning of neural networks containing random variables (such as VAEs), the reparameterization based on random variables is called the reparameterization trick.

The joint probability distribution of the forward process is expressed as

$$q(\boldsymbol{x}_0\boldsymbol{x}_1 \cdots \boldsymbol{x}_T) = q(\boldsymbol{x}_0)\mathcal{N}(\boldsymbol{x}_1; \sqrt{\alpha}_1\boldsymbol{x}_0, \beta_1\boldsymbol{I}) \cdots \mathcal{N}(\boldsymbol{x}_T; \sqrt{\alpha}_T\boldsymbol{x}_{T-1}, \beta_T\boldsymbol{I})$$

The forward process has the following property, stated as a lemma.

Lemma 34.1 *The transition probability distribution from the original sample $\boldsymbol{x}_0$ at step 0 to the sample $\boldsymbol{x}_t$ at step t is the following Gaussian distribution:*

$$q(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_t; \sqrt{\bar{\alpha}_t}\boldsymbol{x}_0, (1 - \bar{\alpha}_t)\boldsymbol{I}) \tag{34.5}$$

where $\bar{\alpha}_t$ satisfies

$$\bar{\alpha}_t = \alpha_1\alpha_2 \cdots \alpha_t$$

Proof Using the random variable representation, expand $\boldsymbol{x}_t$:

$$\begin{aligned}
\boldsymbol{x}_t &= \sqrt{\alpha_t}\boldsymbol{x}_{t-1} + \sqrt{1 - \alpha_t}\boldsymbol{\epsilon}_{t-1} \\
&= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}\boldsymbol{x}_{t-2} + \sqrt{1 - \alpha_{t-1}}\boldsymbol{\epsilon}_{t-2}) + \sqrt{1 - \alpha_{t-1}}\boldsymbol{\epsilon}_{t-1} \\
&= \sqrt{\alpha_t\alpha_{t-1}}\boldsymbol{x}_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\tilde{\boldsymbol{\epsilon}}_{t-2} \\
&= \cdots \\
&= \sqrt{\bar{\alpha}_t}\boldsymbol{x}_0 + \sqrt{1 - \bar{\alpha}_t}\tilde{\boldsymbol{\epsilon}}_0
\end{aligned}$$

where $\boldsymbol{\epsilon}_{t-1}$ and $\boldsymbol{\epsilon}_{t-2}$ follow the standard Gaussian distribution $\boldsymbol{\epsilon}_{t-1} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ and $\boldsymbol{\epsilon}_{t-2} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$. The new random variable $\tilde{\boldsymbol{\epsilon}}_{t-2} = \boldsymbol{\epsilon}_{t-1} + \boldsymbol{\epsilon}_{t-2}$ also follows the standard Gaussian distribution $\tilde{\boldsymbol{\epsilon}}_{t-2} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.

This uses the following property of multivariate Gaussian distributions: the sum of two isotropic Gaussian random variables also follows an isotropic Gaussian distribution,

$$\begin{aligned} \boldsymbol{x}_1 \sim \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_1, \sigma_1^2 \boldsymbol{I}), \quad \boldsymbol{x}_2 \sim \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_2, \sigma_2^2 \boldsymbol{I}) \\ \boldsymbol{x}_1 + \boldsymbol{x}_2 \sim \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_1 + \boldsymbol{\mu}_2, (\sigma_1^2 + \sigma_2^2) \boldsymbol{I}) \end{aligned}$$

Thus $\boldsymbol{x}_t$ can be rewritten as

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t} \boldsymbol{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon} \qquad \blacksquare$$

This property means that the transition probability distribution from the sample $\boldsymbol{x}_0$ at step 0 to the sample $\boldsymbol{x}_t$ at step t can be expressed explicitly. Given the original sample $\boldsymbol{x}_0$, one can directly sample—equivalently, add Gaussian noise—to obtain the sample $\boldsymbol{x}_t$.

This property also implies that as $t \to \infty$, the transition probability distribution $q(\boldsymbol{x}_t | \boldsymbol{x}_{t-1})$ converges to the standard Gaussian distribution. As a result, when T is sufficiently large, the sample $\boldsymbol{x}_T$ is close to Gaussian white noise:

$$\boldsymbol{x}_T \sim \mathcal{N}(\boldsymbol{x}_T; \mathbf{0}, \boldsymbol{I})$$

A common choice is $T = 1000$. In the forward process, the variance coefficient β_t is set to be sufficiently small, $\beta_t \ll 1$, and gradually increases:

$$\beta_1 < \beta_2 < \cdots < \beta_T$$

for example, linearly.

The magnitude of the variance coefficient represents the amount of Gaussian noise added. In the example of Figure 34.2, the Gaussian noise added to samples near the manifold is small in the forward process, while that added near the origin (the mean of the standard Gaussian distribution) is large. Correspondingly, in the reverse process, high-noise denoising is performed near the origin and low-noise denoising near the manifold, which facilitates better learning of the diffusion model. This is also reflected in SMLD.

3. Reverse Process

The reverse process of DDPM is the process of gradually transforming Gaussian white noise $\boldsymbol{x}_T$ into the original sample $\boldsymbol{x}_0$. Let the sample at step T follow the standard Gaussian distribution:

$$\boldsymbol{x}_T \sim \mathcal{N}(\boldsymbol{x}_T; \mathbf{0}, \boldsymbol{I})$$

The transition probability distribution from step t to step $t-1$ ($t = 1, 2, \ldots, T$) is $q(\boldsymbol{x}_{t-1} | \boldsymbol{x}_t)$.

In general, one does not directly compute the transition probability distribution $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$, because this computation is intractable. By Bayes' theorem,

$$q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) \propto q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})q(\boldsymbol{x}_{t-1})$$

where $q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$ is tractable, but $q(\boldsymbol{x}_{t-1})$ is not.

However, the problem becomes simple if $\boldsymbol{x}_0$ is introduced. The conditional transition probability distribution $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$ given $\boldsymbol{x}_0$ can be conveniently computed from the forward process. In fact, this conditional transition probability distribution is also a Gaussian distribution, and its mean and variance can be derived from the forward process. DDPM is implemented based on this property of the reverse process, which is its main technique.

Lemma 34.2 *The conditional transition probability distribution $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$ from step t to step $t-1$ given step 0 is the following Gaussian distribution:*

$$q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0), \tilde{\beta}_t \boldsymbol{I}) \tag{34.6}$$

Its mean is

$$\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\boldsymbol{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{1-\bar{\alpha}_t}\boldsymbol{x}_0$$

and the coefficient of the variance is

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$$

Proof Using Bayes' theorem and the property that the transition probabilities of the forward process are Gaussian distributions, we can derive that the conditional transition probability distribution of the reverse process is also Gaussian and obtain its probability density function.

$$\begin{aligned}
&q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) \\
&= \frac{q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_0)q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}, \boldsymbol{x}_0)}{q(\boldsymbol{x}_t|\boldsymbol{x}_0)} \\
&= \frac{q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_0)q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})}{q(\boldsymbol{x}_t|\boldsymbol{x}_0)} \\
&= \frac{\mathcal{N}(\boldsymbol{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\boldsymbol{x}_0, (1-\bar{\alpha}_{t-1})\boldsymbol{I})\mathcal{N}(\boldsymbol{x}_t; \sqrt{\alpha_t}\boldsymbol{x}_{t-1}, (1-\alpha_t)\boldsymbol{I})}{\mathcal{N}(\boldsymbol{x}_t; \sqrt{\bar{\alpha}_t}\boldsymbol{x}_0, (1-\bar{\alpha}_t)\boldsymbol{I})} \\
&\propto \exp\left\{-\frac{1}{2}\left[\frac{(\boldsymbol{x}_t - \sqrt{\alpha_t}\boldsymbol{x}_{t-1})^2}{\beta_t} + \frac{(\boldsymbol{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}}\boldsymbol{x}_0)^2}{1-\bar{\alpha}_{t-1}} - \frac{(\boldsymbol{x}_t - \sqrt{\bar{\alpha}_t}\boldsymbol{x}_0)^2}{1-\bar{\alpha}_t}\right]\right\} \\
&= \exp\left\{-\frac{1}{2}\left[\frac{\boldsymbol{x}_t^2 - 2\sqrt{\alpha_t}\boldsymbol{x}_t\boldsymbol{x}_{t-1} + \alpha_t\boldsymbol{x}_{t-1}^2}{\beta_t} + \frac{\boldsymbol{x}_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}}\boldsymbol{x}_{t-1}\boldsymbol{x}_0 + \bar{\alpha}_{t-1}\boldsymbol{x}_0^2}{1-\bar{\alpha}_{t-1}} - \frac{(\boldsymbol{x}_t - \sqrt{\bar{\alpha}_t}\boldsymbol{x}_0)^2}{1-\bar{\alpha}_t}\right]\right\} \\
&= \exp\left\{-\frac{1}{2}\left[\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}\right)\boldsymbol{x}_{t-1}^2 - 2\left(\frac{\sqrt{\alpha_t}}{\beta_t}\boldsymbol{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\boldsymbol{x}_0\right)\boldsymbol{x}_{t-1} + C(\boldsymbol{x}_t, \boldsymbol{x}_0)\right]\right\}
\end{aligned}$$

$C(\boldsymbol{x}_t, \boldsymbol{x}_0)$ is a quantity that does not depend on $\boldsymbol{x}_{t-1}$. The variance coefficient of the Gaussian distribution is

$$\begin{aligned}\tilde{\beta}_t &= 1 \Big/ \left(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}\right) \\ &= \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t\end{aligned}$$

and the mean is

$$\begin{aligned}\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0) &= \left(\frac{\sqrt{\alpha_t}}{\beta_t}\boldsymbol{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\boldsymbol{x}_0\right) \Big/ \left(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}\right) \\ &= \left(\frac{\sqrt{\alpha_t}}{\beta_t}\boldsymbol{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\boldsymbol{x}_0\right) \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}(1-\alpha_t) \\ &= \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\boldsymbol{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{1-\bar{\alpha}_t}\boldsymbol{x}_0\end{aligned}$$

■

By Lemma 34.1, $\boldsymbol{x}_0$ can be recovered from $\boldsymbol{x}_t$:

$$\boldsymbol{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}\left(\boldsymbol{x}_t - \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}\right) \tag{34.7}$$

Substituting into $\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0)$ yields a form that depends only on $\boldsymbol{x}_t$:

$$\begin{aligned}\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0) &= \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\boldsymbol{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{1-\bar{\alpha}_t}\frac{1}{\sqrt{\bar{\alpha}_t}}\left(\boldsymbol{x}_t - \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}\right) \\ &= \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{(1-\alpha_t)}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}\right)\end{aligned} \tag{34.8}$$

4. Model: Neural Network

Both the forward and reverse processes are Markov processes, and the transition probability distribution of the forward process is Gaussian (equivalently, adding Gaussian noise). From the transition probability distribution $q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$ of the forward process from step $t-1$ to step t, one can derive both the forward transition probability distribution $q(\boldsymbol{x}_t|\boldsymbol{x}_0)$ from step 0 to step t and the conditional transition probability distribution $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$ of the reverse process from step t to step $t-1$ given step 0. Both of the latter are Gaussian distributions.

The forward process represents a step-by-step noise-adding stochastic process controlled by the hyperparameters β_t ($t = 1, 2, \ldots, T$), which are predetermined. The reverse process represents a step-by-step denoising stochastic process controlled by a learned neural network.

DDPM uses a neural network to represent the reverse transition probability distribution $p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ for $t = 1, 2, \ldots, T$. Assuming this transition probability distribution is Gaussian,

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{x}_{t-1}; \boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t), \boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)) \tag{34.9}$$

its mean and variance are determined by the neural network. The neural network takes the sample $\boldsymbol{x}_t$ and the step number t as inputs and outputs the mean $\boldsymbol{\mu}_{\boldsymbol{\theta}}$ and variance $\boldsymbol{\Sigma}_{\boldsymbol{\theta}}$, with parameters $\boldsymbol{\theta}$.

The joint probability distribution of the reverse process is expressed as

$$p(\boldsymbol{x}_0\boldsymbol{x}_1\cdots\boldsymbol{x}_T) = p(\boldsymbol{x}_T)\mathcal{N}(\boldsymbol{x}_{T-1};\boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_T,T),\boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\boldsymbol{x}_T,T))\cdots\mathcal{N}(\boldsymbol{x}_0;\boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_1,1),\boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\boldsymbol{x}_1,1))$$

Assume the covariance matrix at each step is diagonal:

$$\boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\boldsymbol{x}_t,t) = \sigma_t^2\boldsymbol{I}$$

Figure 34.3 shows the processing at step t in the forward and reverse processes. The forward process (right to left) samples from step $t-1$ to step t according to $q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$, obtaining $\boldsymbol{x}_t$ from $\boldsymbol{x}_{t-1}$. The reverse process (left to right) samples from step t to step $t-1$ according to $p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$, obtaining $\boldsymbol{x}_{t-1}$ from $\boldsymbol{x}_t$. In principle, $p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ should approximate the transition probability distribution $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$. However, since $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ is difficult to compute, DDPM instead makes $p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ approximate the conditional transition probability distribution $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)$.

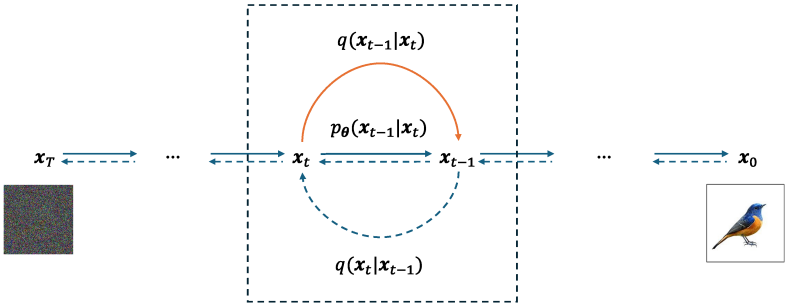

Figure 34.3: Processing at step t in the diffusion forward and reverse processes.

34.2.2 Learning and Generation Algorithms

1. Variational Principle

DDPM uses the variational inference principle to learn a probabilistic generative model from data, analogously to the variational autoencoder (VAE). The forward process corresponds to encoding, and the reverse process corresponds to decoding. The forward process "encodes" the original sample into Gaussian white noise over T steps, and the reverse process "decodes" the Gaussian white noise into the original sample over T steps. The differences from VAEs are as follows: the encoding does not compress the original sample and the decoding does not decompress the Gaussian white noise, meaning that the dimension of the sample vector does not change during the forward and reverse processes. Another difference is that encoding and decoding are performed through two

T-step stochastic processes rather than via two neural networks performing one-step encoding and decoding. Furthermore, the T steps of encoding and decoding in the diffusion process correspond to each other; the encoding process is predetermined and the decoding process must be learned.

As in VAEs, one can define the evidence and evidence lower bound based on the decoder distribution $p_{\boldsymbol{\theta}}(\boldsymbol{x}_0)$:

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_0) \geq \mathbb{E}_{q(\boldsymbol{x}_1\boldsymbol{x}_2\cdots\boldsymbol{x}_T|\boldsymbol{x}_0)}\left[\log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}_0\boldsymbol{x}_1\cdots\boldsymbol{x}_T)}{q(\boldsymbol{x}_1\boldsymbol{x}_2\cdots\boldsymbol{x}_T|\boldsymbol{x}_0)}\right] \tag{34.10}$$

Taking expectations over the original data distribution $q(\boldsymbol{x}_0)$ further yields the expected evidence and expected evidence lower bound:

$$\mathbb{E}_{q(\boldsymbol{x}_0)} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}_0) \geq \mathbb{E}_{q(\boldsymbol{x}_0\boldsymbol{x}_1\cdots\boldsymbol{x}_T)}\left[\log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}_0\boldsymbol{x}_1\cdots\boldsymbol{x}_T)}{q(\boldsymbol{x}_1\boldsymbol{x}_2\cdots\boldsymbol{x}_T|\boldsymbol{x}_0)}\right] \tag{34.11}$$

Directly maximizing the expected evidence to learn the model parameters is computationally intractable. DDPM learns by maximizing the expected evidence lower bound, equivalently minimizing the corresponding loss function $L(\boldsymbol{\theta})$. Expanding the loss function yields the following result, where $p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ approximates $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$:

$$
\begin{aligned}
&L(\boldsymbol{\theta}) \\
&= \mathbb{E}_{q(\boldsymbol{x}_0\boldsymbol{x}_1\cdots\boldsymbol{x}_T)}\left[\log \frac{q(\boldsymbol{x}_1\boldsymbol{x}_2\cdots\boldsymbol{x}_T|\boldsymbol{x}_0)}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_0\boldsymbol{x}_1\cdots\boldsymbol{x}_T)}\right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0\boldsymbol{x}_1\cdots\boldsymbol{x}_T)}\left[\log \frac{\prod_{t=1}^{T} q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_T)\prod_{t=1}^{T} p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)}\right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0\boldsymbol{x}_1\cdots\boldsymbol{x}_T)}\left[-\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_T) + \sum_{t=1}^{T}\log \frac{q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)}\right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0\boldsymbol{x}_1\cdots\boldsymbol{x}_T)}\left[-\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_T) + \sum_{t=2}^{T}\log \frac{q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)} + \log\frac{q(\boldsymbol{x}_1|\boldsymbol{x}_0)}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{x}_1)}\right] \\
&= \mathbb{E}_{q}\left[-\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_T) + \sum_{t=2}^{T}\log \frac{q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)}\cdot\frac{q(\boldsymbol{x}_t|\boldsymbol{x}_0)}{q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_0)} + \log\frac{q(\boldsymbol{x}_1|\boldsymbol{x}_0)}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{x}_1)}\right] \\
&= \mathbb{E}_{q}\left[-\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_T) + \sum_{t=2}^{T}\log \frac{q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)} + \log\frac{q(\boldsymbol{x}_T|\boldsymbol{x}_0)}{q(\boldsymbol{x}_1|\boldsymbol{x}_0)} + \log\frac{q(\boldsymbol{x}_1|\boldsymbol{x}_0)}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{x}_1)}\right] \\
&= \mathbb{E}_{q}\left[\log \frac{q(\boldsymbol{x}_T|\boldsymbol{x}_0)}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_T)} + \sum_{t=2}^{T}\log \frac{q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)} - \log p_{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{x}_1)\right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0,\boldsymbol{x}_T)}\left[\log \frac{q(\boldsymbol{x}_T|\boldsymbol{x}_0)}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_T)}\right] + \sum_{t=2}^{T}\mathbb{E}_{q(\boldsymbol{x}_0,\boldsymbol{x}_{t-1},\boldsymbol{x}_t)}\left[\log \frac{q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)}{p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)}\right] - \mathbb{E}_{q(\boldsymbol{x}_0,\boldsymbol{x}_1)}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{x}_1)\right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0)}\left[\mathrm{KL}\left(q(\boldsymbol{x}_T|\boldsymbol{x}_0)\|p_{\boldsymbol{\theta}}(\boldsymbol{x}_T)\right)\right] + \sum_{t=2}^{T}\mathbb{E}_{q(\boldsymbol{x}_0,\boldsymbol{x}_t)}\left[\mathrm{KL}\left(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)\|p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)\right)\right] \\
&+ \mathbb{E}_{q(\boldsymbol{x}_0,\boldsymbol{x}_1)}\left[-\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{x}_1)\right] \qquad (34.12)
\end{aligned}
$$

The fifth step uses the Markov property $q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1},\boldsymbol{x}_0)$ and Bayes' theorem. The last step introduces KL divergence.

The distribution $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)$ is Gaussian, written as:

$$q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_{t-1};\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t,\boldsymbol{x}_0),\tilde{\beta}_t\boldsymbol{I}) \tag{34.13}$$

$$\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t,\boldsymbol{x}_0) = \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}\right) \tag{34.14}$$

Correspondingly, the distribution $p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ is also Gaussian, written as:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{x}_{t-1};\boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t,t),\sigma_t^2\boldsymbol{I}) \tag{34.15}$$

$$\boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t,t) = \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t,t)\right) \tag{34.16}$$

Assuming that the means of $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$ and $p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ have the same form, and that their variances are equal (setting $\sigma_t^2 = \tilde{\beta}_t$ or $\sigma_t^2 = \beta_t$), the neural network reduces to $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)$, whose input is the sample $\boldsymbol{x}_t$ and step number t, output is the Gaussian noise $\boldsymbol{\epsilon}$, and parameters are $\boldsymbol{\theta}$.

2. Predicting Gaussian Noise

Learning in DDPM is performed in practice by minimizing a simplified loss function, training a neural network to predict Gaussian noise.

Expanding the loss function, the first term L_T is:

$$L_T(\boldsymbol{\theta}) = \mathbb{E}_{q(\boldsymbol{x}_0)} \left[\mathrm{KL} \left(q(\boldsymbol{x}_T|\boldsymbol{x}_0) \| p_{\boldsymbol{\theta}}(\boldsymbol{x}_T) \right) \right] \tag{34.17}$$

This loss L_T is a constant and plays no role in minimization.

The intermediate loss terms L_{t-1} $(t = T, T-1, \ldots, 2)$ are obtained by computing the expected KL divergence between $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$ and $p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$, with the expectation taken over $q(\boldsymbol{x}_0, \boldsymbol{x}_t)$:

$$\begin{aligned}
L_{t-1}(\boldsymbol{\theta}) &= \mathbb{E}_{q(\boldsymbol{x}_0, \boldsymbol{x}_t)} \left[\mathrm{KL} \left(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) \| p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) \right) \right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0) q(\boldsymbol{x}_t|\boldsymbol{x}_0)} \left[\mathrm{KL} \left(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) \| p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) \right) \right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0) q(\boldsymbol{x}_t|\boldsymbol{x}_0)} \left[\frac{1}{2\sigma^2(t)} \| \boldsymbol{\mu}(\boldsymbol{x}_t, \boldsymbol{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) \|^2 \right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0) p(\boldsymbol{\epsilon})} \left[\frac{1}{2\sigma^2(t)} \left\| \frac{1}{\sqrt{\alpha_t}} \left[\boldsymbol{x}_t - \frac{(1-\alpha_t)}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon} \right] - \frac{1}{\sqrt{\alpha_t}} \left[\boldsymbol{x}_t - \frac{(1-\alpha_t)}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) \right] \right\|^2 \right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0) p(\boldsymbol{\epsilon})} \left[\frac{1}{2\sigma^2(t)} \frac{(1-\alpha_t)^2}{\alpha_t (1-\bar{\alpha}_t)} \| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) \|^2 \right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0) p(\boldsymbol{\epsilon})} \left[\frac{1}{2\sigma^2(t)} \frac{(1-\alpha_t)^2}{\alpha_t (1-\bar{\alpha}_t)} \| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\sqrt{\bar{\alpha}_t} \boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t} \boldsymbol{\epsilon}, t) \|^2 \right]
\end{aligned} \tag{34.18}$$

The fourth step involves reparameterization.

The last term L_0 is:

$$L_0(\boldsymbol{\theta}) = \mathbb{E}_{q(\boldsymbol{x}_0, \boldsymbol{x}_1)} \left[-\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{x}_1) \right] \tag{34.19}$$

For the losses L_{t-1}, $t = T, T-1, \ldots, 2$, we drop the coefficients and optimize only the squared loss term. For L_0, we assume the same squared loss optimization. Ignoring L_T, the simplified overall loss function is:

$$\begin{aligned}
L'(\boldsymbol{\theta}) &= \sum_{t=1}^{T} \mathbb{E}_{q(\boldsymbol{x}_0) p(\boldsymbol{\epsilon})} \left[\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) \|^2 \right] \\
&= \sum_{t=1}^{T} \mathbb{E}_{q(\boldsymbol{x}_0) p(\boldsymbol{\epsilon})} \left[\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\sqrt{\bar{\alpha}_t} \boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t} \boldsymbol{\epsilon}, t) \|^2 \right]
\end{aligned} \tag{34.20}$$

The neural network $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)$ predicts the Gaussian noise $\boldsymbol{\epsilon}$ at step t of the forward process. The intuition is as follows: in the forward process, the random transformation from $\boldsymbol{x}_{t-1}$ to $\boldsymbol{x}_t$ at step t is achieved by adding Gaussian noise ϵ; in the reverse process, if one can predict the Gaussian noise ϵ from $\boldsymbol{x}_t$ at step t, then one can also realize the random transformation from $\boldsymbol{x}_t$ to $\boldsymbol{x}_{t-1}$ satisfying the diffusion process requirements (denoising).

Setting the learning objective to predict the original data $\boldsymbol{x}_0$ is theoretically equivalent (see Exercise 34.3). However, experiments show that directly predicting the noise $\boldsymbol{\epsilon}$ achieves better performance.

The transition probability distribution from step t to step $t-1$ of the reverse process can be computed using the learned neural network $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)$. Using the random variable representation:

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) \right) + \sqrt{\beta_t} \boldsymbol{\epsilon} \tag{34.21}$$

Equation (34.21) is called the iteration formula of the DDPM reverse process and is used for data generation. We assume the variance coefficient at each step equals that of the corresponding forward process, $\sigma_t = \sqrt{\beta_t}$. The first term of the iteration formula is deterministic, and the second term is stochastic.

3. Learning and Generation Algorithms

Given a training dataset $\mathcal{T}$, the neural network parameters can be learned by optimizing the loss function (34.20) via gradient descent. The specific algorithms are as follows.

During learning, an original sample $\boldsymbol{x}_0$ is randomly selected, and a step number t is randomly chosen. The sample $\boldsymbol{x}_t$ at step t is obtained from the original sample $\boldsymbol{x}_0$ and Gaussian noise $\boldsymbol{\epsilon}$. The training objective is to predict how to remove the effect of the Gaussian noise $\boldsymbol{\epsilon}$ given $\boldsymbol{x}_t$ and t in order to obtain the sample $\boldsymbol{x}_{t-1}$ at step $t-1$. In other words, the purpose of training is to perform denoising.

Algorithm 34.1 (DDPM — Learning Algorithm)
Input: Training dataset $\mathcal{T}$.
Output: Neural network $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}$.
Hyperparameters: β_t, $t = 1, 2, \ldots, T$.
(1) Initialize the neural network parameters $\boldsymbol{\theta}$.
(2) Repeat until convergence:
(2-1) Sample $\boldsymbol{x}_0$ from the training dataset $\mathcal{T}$;
(2-2) Randomly sample a step number t from $\{T, T-1, \ldots, 1\}$;
(2-3) Randomly sample $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$;
(2-4) Compute the gradient of the loss function and update the neural network parameters:

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\sqrt{\bar{\alpha}_t} \boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t} \boldsymbol{\epsilon}, t) \|^2$$

(3) Output the estimated neural network model. ■

The generation process is the reverse process. The learned neural network is used to perform data generation at each step.

Algorithm 34.2 (DDPM — Generation Algorithm)
Input: Neural network $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}$.
Output: Generated sample $\boldsymbol{x}_0$.
Hyperparameters: β_t, $t = 1, 2, \ldots, T$.
(1) Randomly sample Gaussian white noise $\boldsymbol{x}_T$.
(2) For $(t = T, T-1, \ldots, 1)\{$
(2-1) Randomly sample $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$;
(2-2) If $t = 1$, set $\boldsymbol{\epsilon} = \mathbf{0}$;
(2-3) Compute $\boldsymbol{x}_{t-1}$ from $\boldsymbol{x}_t$:

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) \right) + \sqrt{\beta_t} \boldsymbol{\epsilon}$$

$\}$
(3) Output the generated sample $\boldsymbol{x}_0$. ■

4. Predicting the Score Function

DDPM learning generally uses a neural network to predict the Gaussian noise of the forward process. However, there is an alternative formulation that predicts the score function of the forward process. DDPM and SMLD are connected through score function learning. The following explains the score-function-based learning algorithm for DDPM.

In Bayesian estimation, assume the observed data $\boldsymbol{x}$ is a random variable following a Gaussian distribution:

$$\boldsymbol{x} \sim p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$$

where the mean $\boldsymbol{\mu}$ is also a random variable with a prior distribution, and the variance $\boldsymbol{\Sigma}$ is a constant. Tweedie's formula gives the posterior expectation of the mean $\boldsymbol{\mu}$ given the observed data $\boldsymbol{x}$:

$$\mathbb{E}[\boldsymbol{\mu}|\boldsymbol{x}] = \boldsymbol{x} + \boldsymbol{\Sigma} \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}) \tag{34.22}$$

Here $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$ is the score function, defined in Section 34.3.

The transition probability distribution from sample $\boldsymbol{x}_0$ to sample $\boldsymbol{x}_t$ in the DDPM forward process is

$$q(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_t; \sqrt{\bar{\alpha}_t}\boldsymbol{x}_0, (1-\bar{\alpha}_t)\boldsymbol{I})$$

Applying Tweedie's formula yields the following result.

Lemma 34.3 *In the forward process, the posterior expectation of the mean $\sqrt{\bar{\alpha}_t}\boldsymbol{x}_0$ given the sample $\boldsymbol{x}_t$ is*

$$\mathbb{E}[\sqrt{\bar{\alpha}_t}\boldsymbol{x}_0|\boldsymbol{x}_t] = \boldsymbol{x}_t + (1-\bar{\alpha}_t)\nabla_{\boldsymbol{x}_t} \log q(\boldsymbol{x}_t) \tag{34.23}$$

Note that $\boldsymbol{x}_0$ here is a random variable.

The mean $\sqrt{\bar{\alpha}_t}\boldsymbol{x}_0$ is approximately equal to the posterior expectation:

$$\sqrt{\bar{\alpha}_t}\boldsymbol{x}_0 \approx \boldsymbol{x}_t + (1-\bar{\alpha}_t)\nabla_{\boldsymbol{x}_t}\log q(\boldsymbol{x}_t)$$

and therefore:

$$\boldsymbol{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}\boldsymbol{x}_t + \frac{(1-\bar{\alpha}_t)}{\sqrt{\bar{\alpha}_t}}\nabla_{\boldsymbol{x}_t}\log q(\boldsymbol{x}_t) \tag{34.24}$$

We also obtain the transformation relationship from $\boldsymbol{x}_0$ to $\boldsymbol{x}_t$ in the forward process:

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\boldsymbol{x}_0 - (1-\bar{\alpha}_t)\nabla_{\boldsymbol{x}_t}\log q(\boldsymbol{x}_t) \tag{34.25}$$

Comparing Eq. (34.14) and Eq. (34.25) gives the relationship between the score function and Gaussian noise in DDPM:

$$\nabla_{\boldsymbol{x}_t}\log q(\boldsymbol{x}_t) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon} \tag{34.26}$$

The conditional transition probability distribution $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)$ of the reverse process is Gaussian with mean:

$$\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t,\boldsymbol{x}_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\boldsymbol{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{1-\bar{\alpha}_t}\boldsymbol{x}_0$$

Substituting Eq. (34.24) yields:

$$\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t,\boldsymbol{x}_0) = \frac{1}{\sqrt{\alpha_t}}\left[\boldsymbol{x}_t + (1-\alpha_t)\nabla_{\boldsymbol{x}_t}\log q(\boldsymbol{x}_t)\right] \tag{34.27}$$

The transition probability distribution $p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ is also Gaussian with mean:

$$\boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t,t) = \frac{1}{\sqrt{\alpha_t}}\left[\boldsymbol{x}_t + (1-\alpha_t)\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t,t)\right] \tag{34.28}$$

where $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t,t)$ denotes the neural network.

Defining the variational lower bound and loss function, the loss L_{t-1} is:

$$\begin{aligned}
L_{t-1}(\boldsymbol{\theta}) &= \mathbb{E}_{q(\boldsymbol{x}_0)p(\boldsymbol{\epsilon})}\left[\mathrm{KL}\left(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)\|p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)\right)\right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0)p(\boldsymbol{\epsilon})}\left[\frac{1}{2\sigma_t^2}\|\boldsymbol{\mu}(\boldsymbol{x}_t,\boldsymbol{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t,t)\|^2\right] \\
&= \mathbb{E}_{q(\boldsymbol{x}_0)p(\boldsymbol{\epsilon})}\left[\frac{1}{2\sigma_t^2}\frac{(1-\alpha_t)^2}{\alpha_t(1-\bar{\alpha}_t)}\|\nabla_{\boldsymbol{x}_t}\log q(\boldsymbol{x}_t) - \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t,t)\|^2\right]
\end{aligned} \tag{34.29}$$

This gives an alternative DDPM algorithm where the neural network $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t,t)$ predicts the score function at step t of the forward process. The detailed algorithm is left as an exercise.

The corresponding iteration formula of the reverse process is:

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t + \beta_t\nabla_{\boldsymbol{x}_t}\log q(\boldsymbol{x}_t)\right) + \sqrt{\beta_t}\boldsymbol{\epsilon} \tag{34.30}$$

34.3 Score Matching with Langevin Dynamics

This section presents score matching with Langevin dynamics (SMLD). We first introduce score matching and Langevin dynamics, and then describe the SMLD method.

34.3.1 Score Matching

1. Score Function

Score matching is a learning method for probabilistic generative models (the density function of a probability distribution). We first give the definition of the score function. Suppose the density function of a probability distribution is

$$p(\boldsymbol{x}), \quad \boldsymbol{x} \in \mathbb{R}^d$$

The gradient of the log density function with respect to the data is called the score function:

$$\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$$

The score function represents the trend of change of the probability distribution in the sample space.

The score function $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$ forms a vector field, while the density function $p(\boldsymbol{x})$ forms a scalar field; there is a one-to-one correspondence between them under the normalization condition. The score function can be obtained by differentiating the density function, and the density function can be obtained by integrating the score function.

Figure 34.4 shows the density function and score function of a bivariate Gaussian mixture model, with density function

$$p(\boldsymbol{x}) = \pi_1 \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_1, \boldsymbol{I}) + \pi_2 \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_2, \boldsymbol{I})$$

with $\pi_1 = 0.6$, $\pi_2 = 0.4$, $\boldsymbol{\mu}_1 = (2, 2)^{\mathrm{T}}$, and $\boldsymbol{\mu}_2 = (-2, -2)^{\mathrm{T}}$. The density function is shown by contour lines. The score function is represented by directed line segments; length indicates the magnitude of the score vector and the arrowhead indicates its direction. The model consists of two Gaussian distributions and thus has two peaks. The score vector points in the direction of increasing probability density; it is large on the slopes and approaches zero at the peaks.

Assume the probability distribution $p_{\boldsymbol{\theta}}(\boldsymbol{x})$ is a Gibbs distribution:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \frac{\exp\left(-f_{\boldsymbol{\theta}}(\boldsymbol{x})\right)}{Z_{\boldsymbol{\theta}}} \tag{34.31}$$

where $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ is the energy function with parameters $\boldsymbol{\theta}$, and $Z_{\boldsymbol{\theta}}$ is the normalization term satisfying $\int p_{\boldsymbol{\theta}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = 1$. A special case of the Gibbs distribution is the Boltzmann distribution:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \frac{\exp\left(-U_{\boldsymbol{\theta}}(\boldsymbol{x})/k_B T\right)}{Z_{\boldsymbol{\theta}}} \tag{34.32}$$

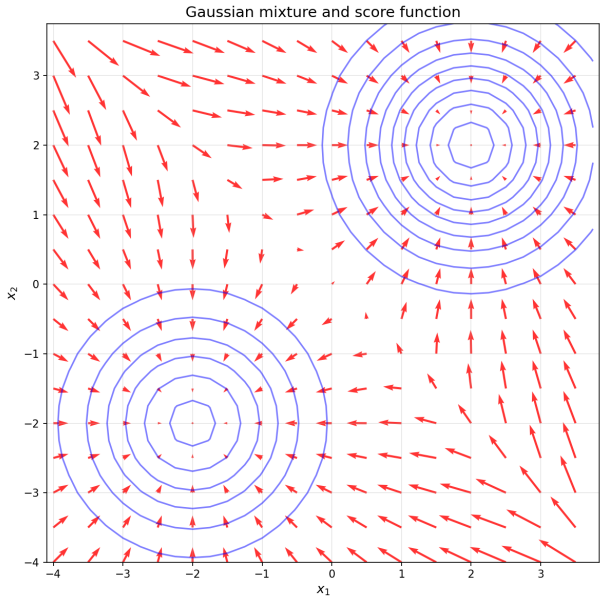

Figure 34.4: Score function.

where $U_{\boldsymbol{\theta}}(\boldsymbol{x})$ is the potential energy function, k_B is the Boltzmann constant, and T is the temperature.

For the Boltzmann distribution, the score function becomes:

$$\nabla_{\boldsymbol{x}} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = -\frac{1}{k_B T} \nabla_{\boldsymbol{x}} U_{\boldsymbol{\theta}}(\boldsymbol{x}) \tag{34.33}$$

This corresponds to the following relationship in physics: the score function represents the vector field of a conservative force (such as gravity), where the vector points in the direction of decreasing potential energy, which is the direction of increasing probability density (Figure 34.4).

2. General Score Matching

Consider the problem of learning a probabilistic generative model and generating data. The training dataset is a random sample from an unknown probability distribution $q(\boldsymbol{x})$:

$$\mathcal{T} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$$

The probability distribution $p_{\boldsymbol{\theta}}(\boldsymbol{x})$ is the model to be learned; it is a Gibbs distribution that approximates $q(\boldsymbol{x})$.

The most direct approach is maximum likelihood estimation, estimating the model parameters by minimizing the log loss function:

$$L(\boldsymbol{\theta}) = \mathbb{E}_{q(\boldsymbol{x})} \left[-\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) \right] \tag{34.34}$$

However, computing the normalization term $Z_{\boldsymbol{\theta}}$ is computationally expensive and often intractable.

The score matching method can circumvent this intractability. Specifically, it sets the learning objective to learn the score function of the probability distribution rather than its density function. The score function (rather than the density function) is used in applications, such as Langevin dynamics discussed later. For the Gibbs distribution, the score function becomes:

$$\nabla_{\boldsymbol{x}} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = -\nabla_{\boldsymbol{x}} f_{\boldsymbol{\theta}}(\boldsymbol{x}) - \nabla_{\boldsymbol{x}} \log Z_{\boldsymbol{\theta}} = -\nabla_{\boldsymbol{x}} f_{\boldsymbol{\theta}}(\boldsymbol{x})$$

One only needs to compute the negative gradient of the energy function with respect to the data, without computing the normalization term. Note that here we take the gradient with respect to the data $\boldsymbol{x}$, not with respect to the parameters $\boldsymbol{\theta}$, so the gradient of the normalization term is zero.

The score matching method specifically uses a neural network to represent the score function:

$$\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = -\nabla_{\boldsymbol{x}} f_{\boldsymbol{\theta}}(\boldsymbol{x}) \tag{34.35}$$

This neural network is learned from data by minimizing the following loss function:

$$L_{\text{esm}}(\boldsymbol{\theta}) = \mathbb{E}_{q(\boldsymbol{x})}\left[\frac{1}{2}\|\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}) - \nabla_{\boldsymbol{x}} \log q(\boldsymbol{x})\|^2\right] \tag{34.36}$$

The loss function L_{esm} is in fact the Fisher divergence. This learning method is called explicit score matching.

Here $\nabla_{\boldsymbol{x}} \log q(\boldsymbol{x})$ is the true value of the score function of the distribution to be learned, which is unknown during training, so the problem seems unsolvable. Interestingly, the optimization problem of score matching can be carried out via an equivalent problem, namely minimizing the following loss function L_{ism}, without needing to know the true score function. This learning method is called implicit score matching:

$$L_{\text{ism}}(\boldsymbol{\theta}) = \mathbb{E}_{q(\boldsymbol{x})}\left[\frac{1}{2}\|\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x})\|^2 + \nabla_{\boldsymbol{x}} \cdot \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x})\right] \tag{34.37}$$

where $\nabla_{\boldsymbol{x}} \cdot$ is the divergence operator.

Theorem 34.1 *Minimizing the implicit score matching objective L_{ism} is equivalent to minimizing the explicit score matching objective L_{esm}:*

$$\min_{\boldsymbol{\theta}} L_{\text{ism}}(\boldsymbol{\theta}) = \min_{\boldsymbol{\theta}} L_{\text{esm}}(\boldsymbol{\theta}) \tag{34.38}$$

This optimization problem still has computational efficiency issues. Computing the second term requires computing the divergence of the score matching model, which has high computational complexity in high dimensions, making implicit score matching impractical.

3. Denoising Score Matching

Denoising score matching adds a small amount of random noise to the original data, learns the score matching function on the noisy data as an estimate of the score matching function on the original data, and simultaneously resolves the computational inefficiency of explicit score matching.

Noise is added using a conditional Gaussian distribution, where σ^2 is the variance coefficient controlling the amount of noise:

$$q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) = \mathcal{N}(\tilde{\boldsymbol{x}}; \boldsymbol{x}, \sigma^2 \boldsymbol{I}) \tag{34.39}$$

Learning minimizes the following squared loss function:

$$L_{\mathrm{dsm}}(\boldsymbol{\theta}) = \mathbb{E}_{q(\boldsymbol{x})q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x})}\left[\frac{1}{2}\|\boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}}) - \nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x})\|^2\right] \tag{34.40}$$

Here the noisy sample $\tilde{\boldsymbol{x}}$ is drawn from the original sample $\boldsymbol{x}$ according to $\mathcal{N}(\tilde{\boldsymbol{x}}; \boldsymbol{x}, \sigma^2 \boldsymbol{I})$. The score function $\nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x})$ has an analytic solution and can be computed efficiently:

$$\nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) = -\frac{\tilde{\boldsymbol{x}} - \boldsymbol{x}}{\sigma^2} \tag{34.41}$$

The learned neural network $\boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})$ thus represents the score matching function $\nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}})$ of the noisy data. The following theorem provides the theoretical guarantee. The theorem holds under more general conditions and does not require the noisy data distribution to be Gaussian.

Theorem 34.2 *Minimizing the denoising score matching objective L_{dsm} is equivalent to minimizing the explicit score matching objective L_{esm}:*

$$\min_{\boldsymbol{\theta}} L_{\mathrm{dsm}}(\boldsymbol{\theta}) = \min_{\boldsymbol{\theta}} L_{\mathrm{esm}}(\boldsymbol{\theta}) \tag{34.42}$$

Proof Expand the objective L_{esm} from its definition:

$$\begin{aligned} L_{\mathrm{esm}}(\boldsymbol{\theta}) &= \mathbb{E}_{q(\tilde{\boldsymbol{x}})}\left[\frac{1}{2}\|\boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}}) - \nabla_{\tilde{\boldsymbol{x}}} \log q(\tilde{\boldsymbol{x}})\|^2\right] \\ &= \mathbb{E}_{q(\tilde{\boldsymbol{x}})}\left[\frac{1}{2}\|\boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})\|^2 - \boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \nabla_{\tilde{\boldsymbol{x}}} \log q(\tilde{\boldsymbol{x}}) + \frac{1}{2}\|\nabla_{\tilde{\boldsymbol{x}}} \log q(\tilde{\boldsymbol{x}})\|^2\right] \end{aligned}$$

The second term can be written as:

$$\begin{aligned} \mathbb{E}_{q(\tilde{\boldsymbol{x}})}\left[\boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \nabla_{\tilde{\boldsymbol{x}}} \log q(\tilde{\boldsymbol{x}})\right] &= \int (\boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \nabla_{\tilde{\boldsymbol{x}}} \log q(\tilde{\boldsymbol{x}})) q(\tilde{\boldsymbol{x}}) \mathrm{d}\tilde{\boldsymbol{x}} \\ &= \int \boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \nabla_{\tilde{\boldsymbol{x}}} q(\tilde{\boldsymbol{x}}) \mathrm{d}\tilde{\boldsymbol{x}} \end{aligned}$$

Using the fact that $q(\tilde{\boldsymbol{x}}) = \int q(\boldsymbol{x}) q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \mathrm{d}\boldsymbol{x}$, one can derive:

$$\begin{aligned}
\int \boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \nabla_{\tilde{\boldsymbol{x}}} q(\tilde{\boldsymbol{x}}) \mathrm{d}\tilde{\boldsymbol{x}} &= \int \boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \nabla_{\tilde{\boldsymbol{x}}} \left(\int q(\boldsymbol{x}) q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \mathrm{d}\boldsymbol{x} \right) \mathrm{d}\tilde{\boldsymbol{x}} \\
&= \int \boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \left(\int q(\boldsymbol{x}) \nabla_{\tilde{\boldsymbol{x}}} q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \mathrm{d}\boldsymbol{x} \right) \mathrm{d}\tilde{\boldsymbol{x}} \\
&= \int \boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \left(\int q(\boldsymbol{x}) q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \mathrm{d}\boldsymbol{x} \right) \mathrm{d}\tilde{\boldsymbol{x}} \\
&= \int \int q(\boldsymbol{x}) q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \left(\boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \right) \mathrm{d}\boldsymbol{x} \mathrm{d}\tilde{\boldsymbol{x}} \\
&= \mathbb{E}_{q(\boldsymbol{x}) q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x})} \left[\boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \right] \qquad (34.43)
\end{aligned}$$

Substituting back into L_{esm}:

$$L_{\mathrm{esm}}(\boldsymbol{\theta}) = \frac{1}{2} \mathbb{E}_{q(\boldsymbol{x}) q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x})} \left[\| \boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}}) \|^2 \right] - \mathbb{E}_{q(\boldsymbol{x}) q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x})} \left[\boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \right] + C_1$$

where the third term is a constant C_1 independent of $\boldsymbol{\theta}$.

Expanding L_{dsm} from its definition:

$$\begin{aligned}
L_{\mathrm{dsm}}(\boldsymbol{\theta}) &= \mathbb{E}_{q(\boldsymbol{x}) q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x})} \left[\frac{1}{2} \| \boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}}) - \nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \|^2 \right] \\
&= \mathbb{E}_{q(\boldsymbol{x}) q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x})} \left[\frac{1}{2} \| \boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}}) \|^2 - \boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) + \frac{1}{2} \| \nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \|^2 \right] \\
&= \mathbb{E}_{q(\boldsymbol{x}) q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x})} \left[\frac{1}{2} \| \boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}}) \|^2 \right] - \mathbb{E}_{q(\boldsymbol{x}) q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x})} \left[\boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}})^{\mathrm{T}} \nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}}|\boldsymbol{x}) \right] + C_2
\end{aligned} \qquad (34.44)$$

where the third term is a constant C_2 independent of $\boldsymbol{\theta}$.

Comparing Eq. (34.43) and Eq. (34.44) gives:

$$L_{\mathrm{dsm}}(\boldsymbol{\theta}) = L_{\mathrm{esm}}(\boldsymbol{\theta}) + C_2 - C_1$$

■

Corollary 34.1 *When the noise coefficient σ is sufficiently small, the optimal score function $\boldsymbol{s}_{\boldsymbol{\theta}^*}(\boldsymbol{x})$ learned by denoising score matching is a good approximation of the true score function $\nabla_{\boldsymbol{x}} \log q(\boldsymbol{x})$:*

$$\boldsymbol{s}_{\boldsymbol{\theta}^*}(\boldsymbol{x}) \approx \nabla_{\boldsymbol{x}} \log q(\boldsymbol{x}) \qquad (34.45)$$

34.3.2 Langevin Dynamics

Langevin dynamics is a stochastic differential equation in physics that describes the random motion of particles in a fluid. The stochastic differential equation form of the (overdamped) Langevin equation is:

$$\mathrm{d}\boldsymbol{x} = -\frac{1}{\gamma} \nabla_{\boldsymbol{x}} U(\boldsymbol{x}) \mathrm{d}t + \sqrt{\frac{2 k_B T}{\gamma}} \mathrm{d}\boldsymbol{w} \qquad (34.46)$$

where $\boldsymbol{x}$ is the position of the particle in space, t is time, γ is the friction coefficient, $U(\boldsymbol{x})$ is the potential energy function, $\nabla_{\boldsymbol{x}} U(\boldsymbol{x})$ represents the conservative force, k_B is the Boltzmann constant, T is the temperature, and $\mathrm{d}\boldsymbol{w}$ is the Wiener process introduced below.

The solution $\boldsymbol{x}(t)$ of the Langevin dynamics equation is a stochastic process. The first term on the right-hand side is a deterministic function, while the second term is a stochastic process; their combined effect determines the trajectory of the particle in the fluid. Theory guarantees that under normal regularity conditions, the fluid system will eventually reach equilibrium after sufficient time, at which point the distribution of particles follows the Boltzmann distribution $p(\boldsymbol{x}) \propto \exp(-U(\boldsymbol{x})/k_B T)$.

Setting $\delta = k_B T/\gamma$ and using $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}) = -\frac{1}{k_B T} \nabla_{\boldsymbol{x}} U(\boldsymbol{x})$, the SDE can be further written as:

$$\mathrm{d}\boldsymbol{x} = \delta \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}) \mathrm{d}t + \sqrt{2\delta} \mathrm{d}\boldsymbol{w} \tag{34.47}$$

Applied to machine learning, Langevin dynamics becomes a random sampling method for probability distributions, also known as the Langevin Monte Carlo method. Assuming the probability distribution is a more general Gibbs distribution $p(\boldsymbol{x}) \propto \exp(-f(\boldsymbol{x}))$ with score function $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}) = -\nabla_{\boldsymbol{x}} f(\boldsymbol{x})$, one can use the score function to sample from the probability distribution and obtain samples from $p(\boldsymbol{x})$.

Starting by sampling from a fixed distribution (e.g., the standard Gaussian distribution) to obtain $\boldsymbol{x}^{(0)}$, one then iteratively samples according to:

$$\boldsymbol{x}^{(l)} = \boldsymbol{x}^{(l-1)} + \delta \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}^{(l-1)}) + \sqrt{2\delta} \boldsymbol{\epsilon}, \quad l = 1, 2, \ldots, L \tag{34.48}$$

where $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ is Gaussian noise and $\delta > 0$ is the step size. This is in fact the iteration formula of the Euler–Maruyama method, the most basic numerical method for solving the SDE (34.47).

During iteration, $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$ drives the sample toward high-probability regions, while $\boldsymbol{\epsilon}$ introduces random perturbations to encourage the sample to explore the entire distribution, avoiding getting stuck at local points. Their combined effect causes the sample to ultimately converge to the distribution $p(\boldsymbol{x})$.

It can be shown that when $\delta \to 0$ and $L \to \infty$, the sampling converges to the target Gibbs distribution $p(\boldsymbol{x})$. In practice, when δ is sufficiently small and L is sufficiently large, $\boldsymbol{x}^{(L)}$ can be approximately regarded as a sample from that distribution.

The Langevin Monte Carlo method only requires knowing $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$ and not $p(\boldsymbol{x})$ itself in order to sample from $p(\boldsymbol{x})$. If a neural network $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x})$ has been learned by score matching and is used in place of $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$ in the above sampling procedure, one obtains a score-matching-based sampling method.

Figure 34.5 shows the result of applying Langevin dynamics sampling to the bivariate Gaussian mixture model in Figure 34.4, with 2000 iterations and 100 samples.

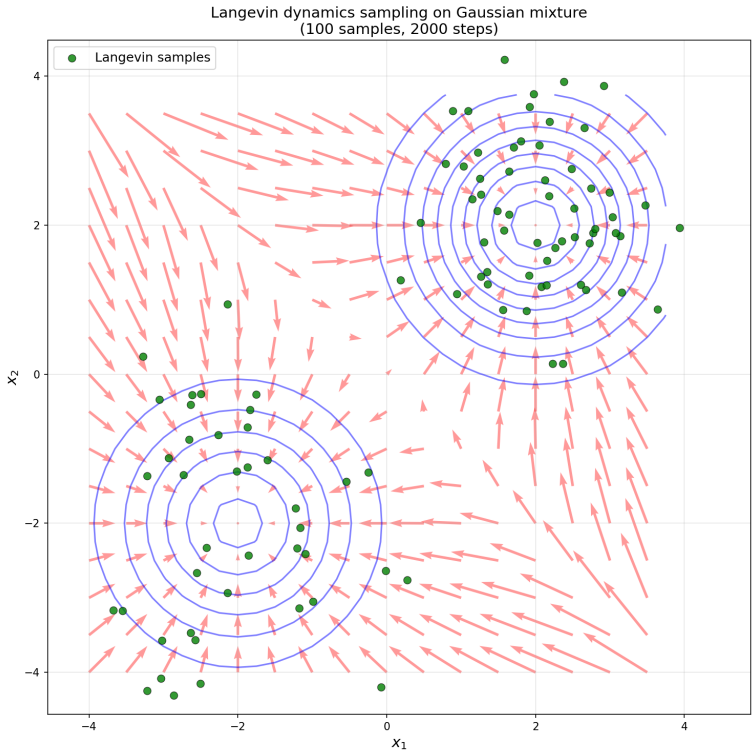

Figure 34.5: Gaussian mixture model and 100 samples from Langevin dynamics sampling.

34.3.3 Learning and Generation Algorithms

We now describe the SMLD method.

The goal is to learn a probabilistic generative model of the data and to sample from the learned probability distribution for data generation. A simple approach is to combine denoising score matching with Langevin dynamics: first learn the score function of the probability distribution via denoising score matching, then use Langevin dynamics for sampling with the learned distribution. However, this approach does not work well in practice for data generation.

SMLD's approach is to add Gaussian noise at different levels to the original data, apply the principles of denoising score matching, and train a single neural network to simultaneously learn the score function of the noisy data distribution at each noise level. The variance coefficient of the Gaussian noise indicates the noise level and is also used as input to the neural network. A smaller variance coefficient corresponds to less noise, and a larger one to more noise. After learning the score functions at different noise levels, Langevin dynamics is used to sample sequentially from the high-noise distribution down to the low-noise

distribution. In theory, the final samples are close to samples from the true distribution. Because sampling proceeds from the high-noise distribution to the low-noise distribution step by step via Langevin dynamics, this sampling method is called annealed Langevin dynamics.

Suppose there are $t = 1, 2, \ldots, T$ noise levels; $T = 1000$ is a common choice. Corresponding to each noise level, there is a conditional Gaussian distribution:

$$\boldsymbol{x}_t \sim q_{\sigma_t}(\boldsymbol{x}_t|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}_t; \boldsymbol{x}, \sigma_t^2 \boldsymbol{I})$$

or equivalently,

$$\boldsymbol{x}_t = \boldsymbol{x} + \sigma_t \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$$

Assume the variance coefficients satisfy

$$\sigma_1 < \sigma_2 < \cdots < \sigma_T$$

With σ_1 sufficiently small, by Corollary 34.1 the following holds:

$$q_{\sigma_1}(\boldsymbol{x}) \approx q(\boldsymbol{x})$$

where $q_{\sigma_1}(\boldsymbol{x}_1) = \int q_{\sigma_1}(\boldsymbol{x}_1|\boldsymbol{x}) q(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}$.

By adding noise with variance coefficient σ_t to the original data $\boldsymbol{x}$ to obtain $\boldsymbol{x}_t$, and learning the neural network $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, \sigma_t)$ for $t = 1, 2, \ldots, T$ from the noisy data, $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_1, \sigma_1)$ is finally used as the score function of the data $\boldsymbol{x}$. Learning minimizes the loss function:

$$L(\boldsymbol{\theta}) = \sum_{t=1}^{T} L_t(\boldsymbol{\theta}) \tag{34.49}$$

where $L_t(\boldsymbol{\theta})$ is the loss at the t-th noise level, defined as:

$$L_t(\boldsymbol{\theta}) = \lambda_t \mathbb{E}_{q(\boldsymbol{x}) q_{\sigma_t}(\boldsymbol{x}_t|\boldsymbol{x})} \left[\| \nabla_{\boldsymbol{x}_t} \log q_{\sigma_t}(\boldsymbol{x}_t|\boldsymbol{x}) - \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, \sigma_t) \|^2 \right] \tag{34.50}$$

where λ_t is a coefficient. Since

$$\nabla_{\boldsymbol{x}_t} \log q_{\sigma_t}(\boldsymbol{x}_t|\boldsymbol{x}) = -\frac{\boldsymbol{x}_t - \boldsymbol{x}}{\sigma_t^2}$$

we have:

$$L_t(\boldsymbol{\theta}) = \lambda_t \mathbb{E}_{q(\boldsymbol{x}) q_{\sigma_t}(\boldsymbol{x}_t|\boldsymbol{x})} \left[\left\| \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, \sigma_t) + \frac{\boldsymbol{x}_t - \boldsymbol{x}}{\sigma_t^2} \right\|^2 \right] \tag{34.51}$$

A common choice is $\lambda_t = \sigma_t^2$. Substituting, the terms inside the norm become $\sigma_t \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, \sigma_t)$ and $(\boldsymbol{x}_t - \boldsymbol{x})/\sigma_t$. Empirically $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, \sigma_t) \propto 1/\sigma_t$ and $(\boldsymbol{x}_t - \boldsymbol{x})/\sigma_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$, so the two terms are of the same order of magnitude.

The learning and generation algorithms for SMLD are given below.

Algorithm 34.3 (SMLD — Learning Algorithm)
Input: Training dataset $\mathcal{T}$.
Output: Neural network $\boldsymbol{s}_{\boldsymbol{\theta}}$.

Hyperparameters: σ_t, $t = 1, 2, \ldots, T$.
(1) Initialize the neural network parameters $\boldsymbol{\theta}$.
(2) Repeat until convergence:
(2-1) Sample $\boldsymbol{x}$ from the training dataset $\mathcal{T}$;
(2-2) For $(t = 1, 2, \ldots, T)\{$
Sample $\boldsymbol{x}_t$ from $q_{\sigma_t}(\boldsymbol{x}_t|\boldsymbol{x})$
$\}$
(2-3) Compute the gradient of the loss function and update the neural network parameters:

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \sum_{t=1}^{T} \sigma_t^2 \left\| \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, \sigma_t) + \frac{\boldsymbol{x}_t - \boldsymbol{x}}{\sigma_t^2} \right\|^2$$

(3) Output the estimated neural network model. ■

Using Langevin dynamics, sampling proceeds from the high-noise level to the low-noise level, i.e., from the T-th noise level down to the 1st noise level. The final sample at the t-th noise level is used as the initial sample for the $(t-1)$-th noise level $(t = T, T-1, \ldots, 1)$, and the final sample at the 1st noise level is used as the output.

Algorithm 34.4 (SMLD — Generation Algorithm)
Input: Neural network $\boldsymbol{s}_{\boldsymbol{\theta}}$.
Output: Generated sample $\boldsymbol{x}$.
Hyperparameters: σ_t $(t = T, T-1, \ldots, 1)$, δ, L.
(1) Randomly sample an initial sample $\boldsymbol{x}^{(0)}$.
(2) For $(t = T, T-1, \ldots, 1)\{$
$\delta_t = \delta \cdot \sigma_t^2 / \sigma_T^2$
For $(l = 1, 2, \ldots, L)\{$
Sample $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$
Update the sample $\boldsymbol{x}^{(l)}$:

$$\boldsymbol{x}^{(l)} \leftarrow \boldsymbol{x}^{(l-1)} + \delta_t \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}^{(l-1)}, \sigma_t) + \sqrt{2\delta_t}\boldsymbol{\epsilon}$$

$\}$
Set $\boldsymbol{x}^{(0)} = \boldsymbol{x}^{(L)}$
$\}$
(3) Output the generated sample $\boldsymbol{x}^{(0)}$. ■

Although sample data exists in a high-dimensional space, it typically lies on a low-dimensional manifold (see Figure 34.2). It has been found that adding only a small amount of noise to the original data on the manifold is insufficient to learn a good generative model. However, by adding noise at different levels to the original data and simultaneously learning denoising at each noise level, a good generative model can be learned. Small perturbations are applied near the manifold and large perturbations near the origin. This is precisely the basic approach of SMLD, corresponding to the noise-adding approach of the DDPM forward process.

Langevin dynamics sampling generally starts from Gaussian white noise (standard Gaussian distribution). Sampling from high noise to low noise can avoid getting trapped in local optima, since high noise is mainly distributed near the origin while low noise is mainly distributed near the manifold.

Comparing Eq. (34.29) and Eq. (34.51), one can see that both DDPM and SMLD learn the score function and have corresponding learning objectives. The closer relationship between the two can be seen through stochastic differential equations below.

34.4 Stochastic Differential Equation Approach

DDPM and SMLD can be viewed as methods for solving stochastic differential equations of different diffusion processes.

1. Stochastic Differential Equations

A stochastic differential equation (SDE) is an equation that characterizes the evolution of a stochastic process. The most typical case is the diffusion process.

For a particle moving in a fluid, if the external force it obeys is deterministic, the change of its position over time can be described by an ordinary differential equation (ODE):

$$\frac{\mathrm{d}\boldsymbol{x}}{\mathrm{d}t} = f(\boldsymbol{x}, t) \tag{34.52}$$

If the particle's motion is also subject to random collision forces, a stochastic differential equation is needed to describe the change of the particle's position.

An SDE contains a deterministic function and a stochastic process:

$$\mathrm{d}\boldsymbol{x} = \boldsymbol{f}(\boldsymbol{x}, t)\mathrm{d}t + \boldsymbol{G}(\boldsymbol{x}, t)\mathrm{d}\boldsymbol{w} \tag{34.53}$$

where $\boldsymbol{x}$ is the particle's position in space, t is time, $\boldsymbol{f}(\boldsymbol{x}, t)$ is the drift coefficient, $\boldsymbol{G}(\boldsymbol{x}, t)$ is the diffusion coefficient, and $\mathrm{d}\boldsymbol{w} = \boldsymbol{\epsilon}\sqrt{\mathrm{d}t}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ is the Wiener process. Note that the drift coefficient is a vector and the diffusion coefficient is a matrix. The first term represents deterministic motion and the second represents random diffusion. Equation (34.53) describes the overall law of the stochastic process.

The Wiener process (or Brownian motion) $\{\boldsymbol{w}_t : t \geq 0\}$ is a stochastic process satisfying the following conditions. $\boldsymbol{w}_t$ is a random variable representing the particle's position at time t. (1) Initial condition: $\boldsymbol{w}_0 = \boldsymbol{0}$. (2) For any sequence of times $0 \leq t_0 < t_1 < \cdots < t_n$, the increments $\boldsymbol{w}_{t_1} - \boldsymbol{w}_{t_0}, \boldsymbol{w}_{t_2} - \boldsymbol{w}_{t_1}, \ldots, \boldsymbol{w}_{t_n} - \boldsymbol{w}_{t_{n-1}}$ are mutually independent random vectors. (3) For any $s \leq t$, the increment $\boldsymbol{w}_t - \boldsymbol{w}_s$ follows a multivariate Gaussian distribution with zero mean vector and covariance matrix $(t-s)\boldsymbol{I}$, i.e., $\boldsymbol{w}_t - \boldsymbol{w}_s \sim \mathcal{N}(\boldsymbol{0}, (t-s)\boldsymbol{I})$.

Langevin dynamics is a specific example of an SDE. In Eq. (34.47), the first term $\delta\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})\mathrm{d}t$ is the deterministic drift term, and the second term $\sqrt{2\delta}\mathrm{d}\boldsymbol{w}$ is the stochastic diffusion term.

In machine learning, a data sample is typically represented as a vector in a high-dimensional space, analogous to a particle. The random trajectory of a

sample in space over time can be described by a stochastic differential equation, and the trajectory is jointly influenced by the deterministic function and the stochastic process.

For an SDE, given appropriate initial conditions and regularity conditions on the coefficients (local Lipschitz continuity, continuity in time, and linear growth conditions), the solution exists and is unique.

In general, the solution to an SDE is a continuous-time Markov process. From Eq. (34.53), the randomness of the sample trajectory comes from the Wiener process. Since the Wiener process has the independent increments property, the state of the sample at any time depends only on the state at the previous time, satisfying the Markov property.

Conversely, a general continuous-time Markov process (in particular, a diffusion process) can also be expressed as the solution to an SDE. The theoretical proof of this statement requires the use of Itô integrals, which we will not expand upon here.

Since SDEs are defined in continuous time, discretizing time yields the corresponding numerical iteration formulas. The most basic method is the Euler–Maruyama method. In fact, the Langevin dynamics sampling algorithm (34.48) is equivalent to using the Euler–Maruyama method.

2. Forward and Reverse Diffusion Processes

Both the forward and reverse processes of the diffusion process can be described by stochastic differential equations. We consider the more specific case where the diffusion coefficient $g(t)$ depends only on time and is a scalar.

Theorem 34.3 (Anderson) *Suppose the forward process of the diffusion process is given by the SDE*

$$\mathrm{d}\boldsymbol{x} = \boldsymbol{f}(\boldsymbol{x}, t)\mathrm{d}t + g(t)\mathrm{d}\boldsymbol{w} \tag{34.54}$$

and assume the relevant functions satisfy appropriate regularity conditions. Then the corresponding reverse-time process satisfies the following reverse-time SDE:

$$\mathrm{d}\boldsymbol{x} = \left[\boldsymbol{f}(\boldsymbol{x}, t) - g^2(t)\nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x})\right] \mathrm{d}t + g(t)\, \mathrm{d}\bar{\boldsymbol{w}}, \tag{34.55}$$

where $\nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x})$ is the score function and $\bar{\boldsymbol{w}}$ is the standard Wiener process in reverse time.

DDPM and SMLD correspond to different forward and reverse SDEs, respectively. Given the forward SDE, the reverse SDE is uniquely determined by the forward SDE. The forward and reverse SDEs of both methods can be solved numerically.

Figure 34.6 shows the forward and reverse processes described by SDEs. In the forward process, both the drift term and the diffusion term are predetermined; in the reverse process, the diffusion term is still known but the drift term contains the unknown score function, which must be obtained through learning. Therefore, the score function is the most critical element in diffusion models.

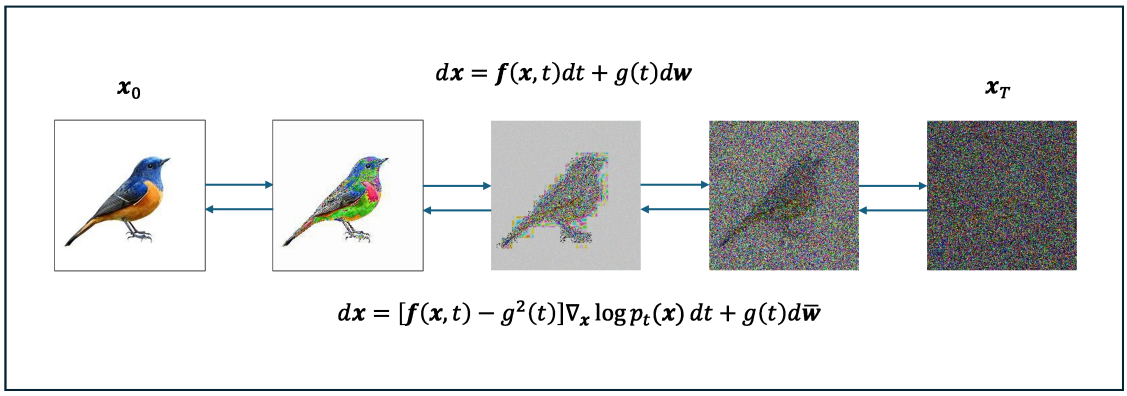

Figure 34.6: Stochastic differential equations describing the forward and reverse processes of the diffusion process.

3. DDPM

Under appropriate approximations, the forward and reverse iteration formulas of DDPM have the same form as the Euler–Maruyama method.

The SDE for the DDPM forward process is:

$$\mathrm{d}\boldsymbol{x} = -\frac{\beta(t)}{2}\boldsymbol{x}\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\boldsymbol{w} \tag{34.56}$$

Applying the Euler–Maruyama method with time step Δt to discretize the SDE at time t:

$$\boldsymbol{x}(t+\Delta t) = \boldsymbol{x}(t) + \left(-\frac{\beta(t)}{2}\boldsymbol{x}(t)\right)\Delta t + \sqrt{\beta(t)}\,(\boldsymbol{w}(t+\Delta t) - \boldsymbol{w}(t))$$

Since the Wiener process increment satisfies

$$\boldsymbol{w}(t+\Delta t) - \boldsymbol{w}(t) = \sqrt{\Delta t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}),$$

the above becomes:

$$\boldsymbol{x}(t+\Delta t) = \left(1 - \frac{\beta(t)}{2}\Delta t\right)\boldsymbol{x}(t) + \sqrt{\beta(t)\Delta t}\,\boldsymbol{\epsilon} \tag{34.57}$$

Integrating the noise rate $\beta(t)$ over the time interval Δt, denoting $\beta_t = \beta(t)\Delta t$, and using a first-order approximation (ignoring $o(\beta_t^2)$ terms):

$$\boldsymbol{x}_t \approx \sqrt{1-\beta_t}\,\boldsymbol{x}_{t-1} + \sqrt{\beta_t}\,\boldsymbol{\epsilon} \tag{34.58}$$

Thus, in the first-order approximation sense, the DDPM forward iteration formula is consistent with the Euler–Maruyama discretization. As $\Delta t \to 0$, the discrete process converges to the corresponding SDE.

By Theorem 34.3, the reverse-time SDE for the DDPM reverse process is:

$$\mathrm{d}\boldsymbol{x} = -\beta(t)\left(\frac{\boldsymbol{x}}{2} + \nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x})\right)\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\bar{\boldsymbol{w}} \tag{34.59}$$

where $\bar{\boldsymbol{w}}$ is the reverse-time Wiener process.

Applying the Euler–Maruyama method with time step Δt to discretize the reverse-time SDE at time t:

$$\begin{aligned}\boldsymbol{x}(t-\Delta t) = \boldsymbol{x}(t) + \beta(t)\left(\frac{\boldsymbol{x}(t)}{2} + \nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x}(t))\right)\Delta t \\ + \sqrt{\beta(t)}\,(\bar{\boldsymbol{w}}(t) - \bar{\boldsymbol{w}}(t-\Delta t))\end{aligned}$$

Since the reverse-time Wiener process increment satisfies

$$\bar{\boldsymbol{w}}(t) - \bar{\boldsymbol{w}}(t-\Delta t) = \sqrt{\Delta t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}),$$

the above becomes:

$$\boldsymbol{x}(t-\Delta t) = \boldsymbol{x}(t) + \beta(t)\left(\frac{\boldsymbol{x}(t)}{2} + \nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x}(t))\right)\Delta t + \sqrt{\beta(t)\Delta t}\,\boldsymbol{\epsilon} \tag{34.60}$$

Denoting $\beta_t = \beta(t)\Delta t$:

$$\boldsymbol{x}_{t-1} = \left(1 + \frac{\beta_t}{2}\right)\boldsymbol{x}_t + \beta_t \nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x}_t) + \sqrt{\beta_t}\,\boldsymbol{\epsilon} \tag{34.61}$$

Using a first-order approximation and ignoring $o(\beta_t^2)$ terms:

$$\boldsymbol{x}_{t-1} \approx \frac{1}{\sqrt{1-\beta_t}}\left(\boldsymbol{x}_t + \beta_t \nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x}_t)\right) + \sqrt{\beta_t}\,\boldsymbol{\epsilon} \tag{34.62}$$

Under this condition, this iteration form has the same form as the score-function-based DDPM reverse iteration formula (34.30).

4. SMLD

SMLD was derived from the perspective of score matching, but it can also be viewed as diffusion-process-based. In fact, adding noise at different levels to the original data corresponds to the forward process, with the noise level corresponding to the diffusion step.

Suppose there exists a forward Markov process where the transition probability distribution from $\boldsymbol{x}_{t-1}$ to $\boldsymbol{x}_t$ is the following Gaussian:

$$q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = \mathcal{N}(\boldsymbol{x}_t; \boldsymbol{x}_{t-1}, (\sigma_t^2 - \sigma_{t-1}^2)\boldsymbol{I}) \tag{34.63}$$

$$\boldsymbol{x}_t = \boldsymbol{x}_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2}\boldsymbol{\epsilon} \tag{34.64}$$

with $\boldsymbol{x}_0 = \boldsymbol{x}$ and $\sigma_0 = 0$.

The mean and variance of $\boldsymbol{x}_t$ $(t = 1, 2, \ldots, T)$ satisfy:

$$\mathbb{E}(\boldsymbol{x}_t) = \mathbb{E}(\boldsymbol{x}_{t-1}) = \cdots = \mathbb{E}(\boldsymbol{x}_1) = \boldsymbol{x}_0$$

$$\begin{aligned}\mathrm{Var}(\boldsymbol{x}_t) &= \mathrm{Var}(\boldsymbol{x}_{t-1}) + (\sigma_t^2 - \sigma_{t-1}^2)\boldsymbol{I} \\ &= \mathrm{Var}(\boldsymbol{x}_{t-2}) + (\sigma_t^2 - \sigma_{t-2}^2)\boldsymbol{I} \\ &= \cdots \\ &= \mathrm{Var}(\boldsymbol{x}_0) + \sigma_t^2 \boldsymbol{I}\end{aligned}$$

This gives the transition probability distribution from $\boldsymbol{x}_0$ to $\boldsymbol{x}_t$:

$$q(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_t; \boldsymbol{x}_0, \sigma_t^2 \boldsymbol{I}) \tag{34.65}$$

which is equivalent to the t-th noise level conditional Gaussian distribution in SMLD. Equation (34.64) is called the SMLD forward process iteration formula.

The SDE corresponding to the SMLD forward process is:

$$\mathrm{d}\boldsymbol{x} = \sqrt{\frac{\mathrm{d}(\sigma^2(t))}{\mathrm{d}t}}\,\mathrm{d}\boldsymbol{w}, \tag{34.66}$$

where $\sigma^2(t)$ is the noise intensity function varying with time. Applying the Euler–Maruyama method and making certain approximations yields an iteration formula of the same form as the SMLD forward process:

$$\boldsymbol{x}_t = \boldsymbol{x}_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2}\,\boldsymbol{\epsilon}. \tag{34.67}$$

By Theorem 34.3, the reverse-time SDE for the SMLD reverse process is:

$$\mathrm{d}\boldsymbol{x} = -\frac{\mathrm{d}(\sigma^2(t))}{\mathrm{d}t}\nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x})\,\mathrm{d}t + \sqrt{\frac{\mathrm{d}(\sigma^2(t))}{\mathrm{d}t}}\,\mathrm{d}\bar{\boldsymbol{w}}, \tag{34.68}$$

where $\bar{\boldsymbol{w}}$ is the reverse-time Wiener process. Applying the Euler–Maruyama method and making certain approximations yields an iteration formula nearly identical in form to the SMLD reverse process:

$$\boldsymbol{x}_{t-1} = \boldsymbol{x}_t + (\sigma_t^2 - \sigma_{t-1}^2)\nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x}_t) + \sqrt{\sigma_t^2 - \sigma_{t-1}^2}\,\boldsymbol{\epsilon}, \tag{34.69}$$

which is also the annealed Langevin dynamics iteration formula.

Thus, the forward and reverse process iteration formulas of both DDPM and SMLD can be viewed as approximation algorithms for solving the stochastic differential equations of different diffusion processes.

34.5 Image Generation

Both DDPM and SMLD have been applied to tasks such as image generation, each with its own advantages on different tasks. Currently, DDPM is the more commonly used model for image generation.

34.5.1 Diffusion Models for Image Generation

Image generation is the task of learning a probabilistic distribution model of images from training data, sampling from the learned model, and automatically generating new image data. The generative adversarial network (GAN), variational autoencoder (VAE), and diffusion model (DM) introduced in this book can all be used for image generation. Currently, diffusion models, especially DDPM, are the primary technique for image generation. The results of image generation are generally evaluated by metrics such as sharpness, realism, plausibility, and diversity. Images generated by diffusion models have reached the point where they are difficult to distinguish from real images.

Figure 34.7 compares the learning and generation workflows of GANs, VAEs, and diffusion models. The characteristic feature of diffusion models is that both the noise-adding and denoising (encoding and decoding) processes are realized through multiple steps of random transformations, with the denoising or decoding performed by neural networks. VAEs directly perform these two processes using neural networks. GANs use a generator network for denoising or decoding and a discriminator network for evaluation. The learning and generation approach of diffusion models has several advantages: (1) the distribution of sample data, such as image data, is complex, and multi-step learning facilitates more accurate learning of the sample distribution; (2) multi-step learning reduces computational complexity, making it feasible to learn complex models; (3) the data generation process is also more stable, and the realism and diversity of the generated data are higher.

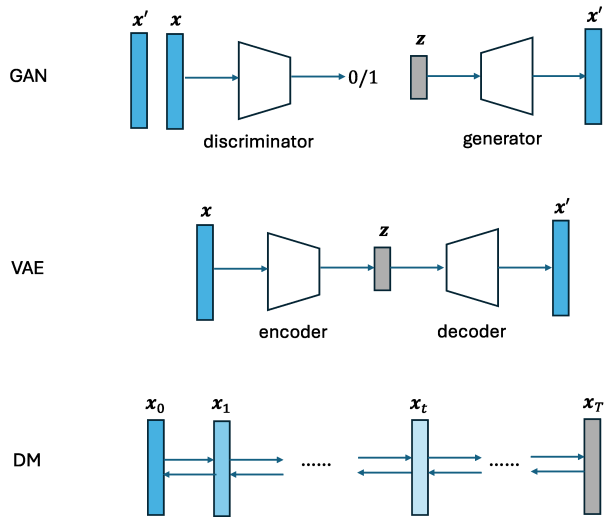

Figure 34.7: Comparison of probabilistic generative models: GAN, VAE, and DM.

DDPM for image generation generally uses UNet or Transformer as the neu-

ral network. Model learning is typically performed not in the original sample space but in the latent space. Image generation is often conditioned on text or other information. Two related techniques—latent-space generation and conditional diffusion models—are introduced below.

34.5.2 Generation in Latent Space

The most direct approach to applying diffusion models to image generation is to learn and use the diffusion model in the sample space of image data. However, this requires large computational resources and affects learning efficiency and generation speed. A common approach is to compress the images into latent-space representations and perform diffusion model learning and generation in the latent space. A widely used method is the latent diffusion model (LDM).

LDM learning proceeds in two steps: Step 1, learn a VAE from the training data to obtain an encoder and decoder, and simultaneously compress the training data into the latent space; Step 2, learn DDPM from the compressed images in the latent space. LDM generation also proceeds in two steps: Step 1, use the learned DDPM to generate a compressed image in the latent space; Step 2, use the decoder to transform the generated compressed image into an image.

The sample space is $\boldsymbol{x} \in \mathbb{R}^{H \times W \times 3}$, where H and W are the height and width of the image and 3 is the number of channels. The latent space is $\boldsymbol{z} \in \mathbb{R}^{h \times w \times c}$, where h and w are the height and width of the compressed image and c is the number of channels. Assume $H/h = W/w = 2^m$, where m is a hyperparameter, typically $m = 4 \sim 8$. Since the dimensionality of the latent space is much smaller than that of the sample space, LDM achieves higher learning efficiency.

Image data generally contains a certain amount of redundancy, so after compression, learning a diffusion model in the latent space still produces new image data with good realism, even though the representations of samples in the latent space are not humanly interpretable. Therefore, one can regard the original image space and the compressed latent space as essentially equivalent in terms of visual representation.

LDM uses UNet as the neural network for DDPM. UNet is a convolutional neural network that uses skip connections. In UNet, the input is first downsampled and then upsampled: the intermediate hidden layers first narrow from wide to narrow and then widen from narrow to wide, while the feature dimensions of the input and output remain unchanged. Also, the samples in the LDM latent space are 2D compressed images. The VAE learning does not use a squared loss function but instead uses image-processing-related loss functions; the DDPM learning loss function remains the squared loss (34.20). LDM achieves a good balance between image generation quality and efficiency, and is currently the primary method used for image generation.

34.5.3 Conditional Generation

Image generation is often performed under given descriptions of categories, content, etc., requiring the generated image to follow language instructions. This is

called conditional generation, as opposed to unconditional generation. Representative methods for conditional generation include classifier-guided diffusion and classifier-free guidance.

1. Classifier-Guided Diffusion

Classifier-guided diffusion uses training data $(\boldsymbol{x}, y)$, where $\boldsymbol{x}$ represents the image and y represents the text. Assume the joint probability distribution of the noisy data $\boldsymbol{x}_t$ at step t ($t = 1, 2, \ldots, T$) and the text y is determined by the diffusion model and classifier:

$$p(\boldsymbol{x}_t, y) = p_{\boldsymbol{\theta}}(\boldsymbol{x}_t) p_{\boldsymbol{\phi}}(y|\boldsymbol{x}_t) \tag{34.70}$$

where $p_{\boldsymbol{\theta}}(\boldsymbol{x}_t)$ is the diffusion model (DDPM), $p_{\boldsymbol{\phi}}(y|\boldsymbol{x}_t)$ is the classifier, and $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$ are the respective parameters.

The score function of the joint distribution is:

$$\begin{aligned}
\nabla_{\boldsymbol{x}_t} \log p(\boldsymbol{x}_t, y) &= \nabla_{\boldsymbol{x}_t} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}_t) + \nabla_{\boldsymbol{x}_t} \log p_{\boldsymbol{\phi}}(y|\boldsymbol{x}_t) \\
&= -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) + \nabla_{\boldsymbol{x}_t} \log p_{\boldsymbol{\phi}}(y|\boldsymbol{x}_t) \\
&= -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \left(\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) - \sqrt{1-\bar{\alpha}_t} \nabla_{\boldsymbol{x}_t} \log p_{\boldsymbol{\phi}}(y|\boldsymbol{x}_t) \right)
\end{aligned} \tag{34.71}$$

Here we use the relationship between the score function and Gaussian noise (34.26).

During learning, the score function of the joint distribution is used. At each step, the noise prediction $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)$ is adjusted by the classifier score function $\nabla_{\boldsymbol{x}_t} \log p_{\boldsymbol{\phi}}(y|\boldsymbol{x}_t)$ to obtain a new noise prediction:

$$\bar{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) = \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) - \lambda \sqrt{1-\bar{\alpha}_t} \nabla_{\boldsymbol{x}_t} \log p_{\boldsymbol{\phi}}(y|\boldsymbol{x}_t) \tag{34.72}$$

where the coefficient λ is a hyperparameter controlling the degree of influence of the classifier on noise prediction. When $\lambda = 0$, the method reduces to DDPM; when $\lambda \neq 0$, the DDPM denoising prediction is adjusted by the classifier's score function.

A drawback of classifier-guided diffusion is that it requires training two neural networks: the diffusion model and the classifier. Classifier-free guidance can accomplish the task of conditional generation with only one neural network.

2. Classifier-Free Guidance

Classifier-free guidance also uses training data $(\boldsymbol{x}, y)$. A single unified neural network is used to learn both the conditional diffusion model $p_{\boldsymbol{\theta}}(\boldsymbol{x}|y)$ and the unconditional diffusion model $p_{\boldsymbol{\theta}}(\boldsymbol{x})$. The network for the conditional model is

$$\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y)$$

and the unconditional model's network is a special case of the conditional model's network:

$$\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) = \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, \phi)$$

where ϕ represents a constant, such as the zero vector.

Based on the same derivation as classifier-guided diffusion, the noise prediction method using the unified network in DDPM learning is:

$$\bar{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y) = \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y) - \lambda\sqrt{1-\bar{\alpha}_t}\nabla_{\boldsymbol{x}_t}\log p_{\boldsymbol{\theta}}(y|\boldsymbol{x}_t) \tag{34.73}$$

where $\nabla_{\boldsymbol{x}_t}\log p_{\boldsymbol{\theta}}(y|\boldsymbol{x}_t)$ is the implicit classifier's score function.

By Bayes' theorem:

$$\begin{aligned}\nabla_{\boldsymbol{x}_t}\log p_{\boldsymbol{\theta}}(y|\boldsymbol{x}_t) &= \nabla_{\boldsymbol{x}_t}\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_t|y) - \nabla_{\boldsymbol{x}_t}\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_t) \\ &= -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\left(\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y) - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\right)\end{aligned} \tag{34.74}$$

Substituting into Eq. (34.74):

$$\begin{aligned}\bar{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y) &= \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y) - \lambda\sqrt{1-\bar{\alpha}_t}\nabla_{\boldsymbol{x}_t}\log p_{\boldsymbol{\theta}}(y|\boldsymbol{x}_t) \\ &= \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y) + \lambda\left(\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y) - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\right) \\ &= (1+\lambda)\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y) - \lambda\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\end{aligned} \tag{34.75}$$

Thus the unified network $\bar{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y)$ is used during learning.

When $\lambda = -1$, this reduces to the unconditional model; when $\lambda = 0$, it reduces to the conditional model. When $\lambda > 0$, it enhances the conditional model's noise prediction while suppressing the unconditional model's noise prediction, making the generated samples more strongly correlated with the given condition. This is the main purpose of using classifier-free guidance in conditional generation: adjusting the strength of the condition.

Neural network training can combine the conditional and unconditional models in a unified manner. For example, a certain proportion of the training data has y set to ϕ, and this proportion is a hyperparameter.

Chapter Summary

1. Basic Idea of Diffusion Models Diffusion models are probabilistic generative models defined based on the diffusion process. The diffusion process consists of a forward process and a reverse process. The forward process gradually transforms data into Gaussian white noise; the reverse process gradually transforms Gaussian white noise into data. Both the forward and reverse processes are Markov processes. The forward process is defined in advance, and model learning takes place in the reverse process.

Representative methods include the denoising diffusion probabilistic model (DDPM) and the score matching with Langevin dynamics model (SMLD).

2. Basic Idea of DDPM The DDPM forward process starts from the original data $\boldsymbol{x}_0$, gradually adds noise over $t = 1, 2, \ldots, T$ steps until reaching approximately Gaussian white noise $\boldsymbol{x}_T$. The reverse process starts from $\boldsymbol{x}_T$ and gradually removes noise over $T-1, \ldots, 1, 0$ steps to recover the original data $\boldsymbol{x}_0$.

The learning objective is to train a neural network to predict the noise added at each step of the forward process.

3. Forward and Reverse Processes of DDPM The DDPM forward process is a Markov process with transition probability:

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \mathcal{N}\big(\boldsymbol{x}_t; \sqrt{\alpha_t}\boldsymbol{x}_{t-1}, \beta_t \boldsymbol{I}\big), \qquad \alpha_t = 1 - \beta_t.$$

The reverse process is also a Markov process. From the forward process, the conditional distribution can be computed:

$$q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) = \mathcal{N}\big(\boldsymbol{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0), \tilde{\beta}_t \boldsymbol{I}\big).$$

DDPM learns based on the variational principle, by maximizing the lower bound of the log evidence $\log p(\boldsymbol{x}_0)$.

The reverse process transition probability distribution is assumed to be Gaussian and parameterized by a neural network:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t) = \mathcal{N}\big(\boldsymbol{x}_{t-1}; \boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t), \boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\big).$$

4. Variational Objective of DDPM The expected evidence lower bound (ELBO) satisfies:

$$\mathbb{E}_{q(\boldsymbol{x}_0)} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}_0) \geq \mathbb{E}_{q(\boldsymbol{x}_{0:T})} \left[\log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}_{0:T})}{q(\boldsymbol{x}_{1:T} \mid \boldsymbol{x}_0)} \right].$$

The main loss term is:

$$L_{t-1}(\boldsymbol{\theta}) = \mathbb{E}\left[\mathrm{KL}\big(q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) \parallel p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)\big) \right], \quad t = 2, \ldots, T.$$

Under fixed covariance, this loss simplifies to the noise prediction error:

$$L'_{t-1}(\boldsymbol{\theta}) = \mathbb{E}\big[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\|^2\big].$$

The forward process can be written as:

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\boldsymbol{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}),$$

where $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$.

The reverse sampling process is:

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) \right) + \sqrt{\beta_t}\boldsymbol{\epsilon}.$$

The relationship between the score function and noise is:

$$\nabla_{\boldsymbol{x}_t} \log q(\boldsymbol{x}_t) = -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}.$$

5. Basic Idea of SMLD SMLD directly learns and uses the score function. During learning, Gaussian noise $q_{\sigma_t}(\boldsymbol{x}_t \mid \boldsymbol{x})$ of varying levels is added to the original data $\boldsymbol{x}$, and a neural network is trained to fit the score function $\nabla_{\boldsymbol{x}_t} \log q_{\sigma_t}(\boldsymbol{x}_t \mid \boldsymbol{x})$ of the data distribution at each noise level σ_t.

During generation, Langevin dynamics is used for sampling, progressively reducing the noise level until generating samples close to the original data distribution.

6. Score Matching of SMLD The score function is defined as:

$$\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}).$$

Score matching uses a neural network $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x})$ to represent the score function and minimizes:

$$L(\boldsymbol{\theta}) = \mathbb{E}_{q(\boldsymbol{x})}\left[\frac{1}{2}\|\nabla_{\boldsymbol{x}} \log q(\boldsymbol{x}) - \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x})\|^2\right].$$

Denoising score matching uses the noisy distribution:

$$q_\sigma(\tilde{\boldsymbol{x}} \mid \boldsymbol{x}) = \mathcal{N}(\tilde{\boldsymbol{x}}; \boldsymbol{x}, \sigma^2 \boldsymbol{I}),$$

with loss function:

$$L(\boldsymbol{\theta}) = \mathbb{E}\left[\frac{1}{2}\|\boldsymbol{s}_{\boldsymbol{\theta}}(\tilde{\boldsymbol{x}}) - \nabla_{\tilde{\boldsymbol{x}}} \log q_\sigma(\tilde{\boldsymbol{x}} \mid \boldsymbol{x})\|^2\right].$$

7. Langevin Dynamics Langevin dynamics is used to sample from a target distribution:

$$\boldsymbol{x}^{(l)} = \boldsymbol{x}^{(l-1)} + \delta \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}^{(l-1)}) + \sqrt{2\delta}\boldsymbol{\epsilon},$$

and the sample distribution converges to $p(\boldsymbol{x})$ as $\delta \to 0$ and $L \to \infty$.

8. Learning and Sampling of SMLD SMLD uses conditional distributions at different noise levels:

$$q_{\sigma_t}(\boldsymbol{x}_t \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}_t; \boldsymbol{x}, \sigma_t^2 \boldsymbol{I}),$$

and learns the conditional score model $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, \sigma_t)$ with loss:

$$L_t(\boldsymbol{\theta}) = \lambda_t \mathbb{E}\left[\left\|\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, \sigma_t) + \frac{\boldsymbol{x}_t - \boldsymbol{x}}{\sigma_t^2}\right\|^2\right].$$

During generation, Langevin dynamics sampling is applied sequentially from high to low noise levels.

9. Stochastic Differential Equation (SDE) Perspective Both DDPM and SMLD can be viewed as discrete-time forms of continuous-time stochastic differential equations (SDEs). Their forward and reverse update formulas correspond to numerical solutions of SDEs.

10. Latent Diffusion Model The latent diffusion model first trains a variational autoencoder (VAE) to map data to the latent space, then trains DDPM in the latent space. During generation, samples are first generated in the latent space and then transformed back to the data space by the decoder.

11. Conditional Generation and Guidance Classifier-guided diffusion adjusts noise prediction using the classifier score function:

$$\bar{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}} = \boldsymbol{\epsilon}_{\boldsymbol{\theta}} - \lambda\sqrt{1-\bar{\alpha}_t}\nabla_{\boldsymbol{x}_t}\log p_{\boldsymbol{\phi}}(y \mid \boldsymbol{x}_t).$$

Classifier-free guidance uses a unified model:

$$\bar{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}} = (1+\lambda)\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y) - \lambda\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t).$$

Further Reading

The main papers on diffusion models include those introducing DDPM and SMLD [1]–[3]. Papers related to SMLD include [4] and [5]. Reference [6] explores the relationship between stochastic differential equations and diffusion models. DDIM (Denoising Diffusion Implicit Model) [7] further improves the generation efficiency of DDPM. The widely used latent diffusion model and UNet for image generation were proposed in [8] and [9], respectively. The ideas of classifier-guided diffusion and classifier-free guidance can be found in [10] and [11]. For a more comprehensive understanding of diffusion models, see the surveys and introductory articles [12–15].

Exercises

34.1 In the forward process of DDPM, as $t \to \infty$, the transition probability distribution $q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$ converges to the standard Gaussian distribution. Explain the reason.

34.2 Prove that the following relation holds in DDPM:

$$\mathbb{E}_{q(\boldsymbol{x}_t)}\left[\mathrm{KL}\left(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)\|p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)\right)\right] = \mathbb{E}_{q(\boldsymbol{x}_0,\boldsymbol{x}_t)}\left[\mathrm{KL}\left(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)\|p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)\right)\right]+C$$

where C is a constant independent of $\boldsymbol{\theta}$. Hint: use the techniques from the DDPM variational inference derivation.

34.3 The DDPM neural network can also be designed to predict the original data $\boldsymbol{x}_0$, which is equivalent to predicting the Gaussian noise $\boldsymbol{\epsilon}$. Derive the iteration formulas for the reverse and forward processes in this case, and explain their equivalence.

34.4 Organize the score-function-based learning and generation algorithms of DDPM and list the differences from SMLD.

34.5 Prove Tweedie's formula for the univariate Gaussian distribution $x \sim p(x) = \mathcal{N}(x;\mu,\sigma^2)$, where the data x is a random variable, the mean μ is a random variable, and the variance σ^2 is a constant:

$$\mathbb{E}[\mu|x] = x + \sigma^2\frac{\mathrm{d}}{\mathrm{d}x}\log p(x)$$

34.6 Implement the Langevin dynamics sampling algorithm and apply it to sample from the following Gaussian mixture model:

$$p(x) = \pi_1 \mathcal{N}(x; \mu_1, \sigma_1^2) + \pi_2 \mathcal{N}(x; \mu_2, \sigma_2^2)$$

with $\pi_1 = 0.6$, $\pi_2 = 0.4$, $\mu_1 = 5$, $\mu_2 = -4$, $\sigma_1 = 0.5$, $\sigma_2 = 0.4$.

34.7 For the Gaussian mixture model in Exercise 34.6, use the DDPM formulas (34.4) and (34.21) to iterate the forward and reverse processes, and plot the random evolution trajectories of 100 samples over time.

34.8 Verify that the SMLD forward and reverse iteration formulas are consistent with the Euler–Maruyama numerical solutions of the corresponding SDEs in the continuous-time limit.

34.9 Derive the score-function form of classifier-free guidance:

$$\nabla_{\boldsymbol{x}_t} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}_t|y) = \gamma \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t, y) + (1-\gamma)\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)$$

where γ is a coefficient.

References

]

[1 SOHL-DICKSTEIN J, WEISS E, MAHESWARANATHAN N, et al. Deep unsupervised learning using nonequilibrium thermodynamics[C]//Proceedings of the 32nd International Conference on Machine Learning. PMLR, 2015: 2256–2265.

[2 SONG Y, ERMON S. Generative modeling by estimating gradients of the data distribution[J]. Advances in Neural Information Processing Systems, 2019, 32.

[3 HO J, JAIN A, ABBEEL P. Denoising diffusion probabilistic models[J]. Advances in Neural Information Processing Systems, 2020, 33: 6840–6851.

[4 HYVÄRINEN A. Estimation of non-normalized statistical models by score matching[J]. Journal of Machine Learning Research, 2005, 6(4): 695–709.

[5 VINCENT P. A connection between score matching and denoising autoencoders[J]. Neural Computation, 2011, 23(7): 1661–1674.

[6 SONG Y, SOHL-DICKSTEIN J, KINGMA D P, et al. Score-based generative modeling through stochastic differential equations[C]//International Conference on Learning Representations, 2021.

[7 SONG J, MENG C, ERMON S. Denoising diffusion implicit models[C]//International Conference on Learning Representations, 2021.

[8 ROMBACH R, BLATTMANN A, LORENZ D, et al. High-resolution image synthesis with latent diffusion models[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 10684–10695.

[9 RONNEBERGER O, FISCHER P, BROX T. U-Net: Convolutional networks for biomedical image segmentation[C]//Medical Image Computing and Computer-Assisted Intervention. Springer, 2015: 234–241.

[10 NICHOL A Q, DHARIWAL P. Improved denoising diffusion probabilistic models[C]//International Conference on Machine Learning. PMLR, 2021: 8162–8171.

[11 HO J, SALIMANS T. Classifier-free diffusion guidance[C]//NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications, 2021.

[12 WENG L. What are diffusion models? Lil'Log, https://lilianweng.github.io/posts/2021-07-11-diffusion-models/, 2021.

[13 LUO C. Understanding diffusion models: A unified perspective. arXiv preprint, arXiv:2208.11970, 2022.
[14 CHAN S H. Tutorial on diffusion models for imaging and vision. arXiv preprint, arXiv:2403.18103, 2024.
[15 LAI C H, SONG Y, KIM D, MITSUFUJI Y, ERMON S. The principles of diffusion models. arXiv preprint arXiv:2510.21890, 2025.